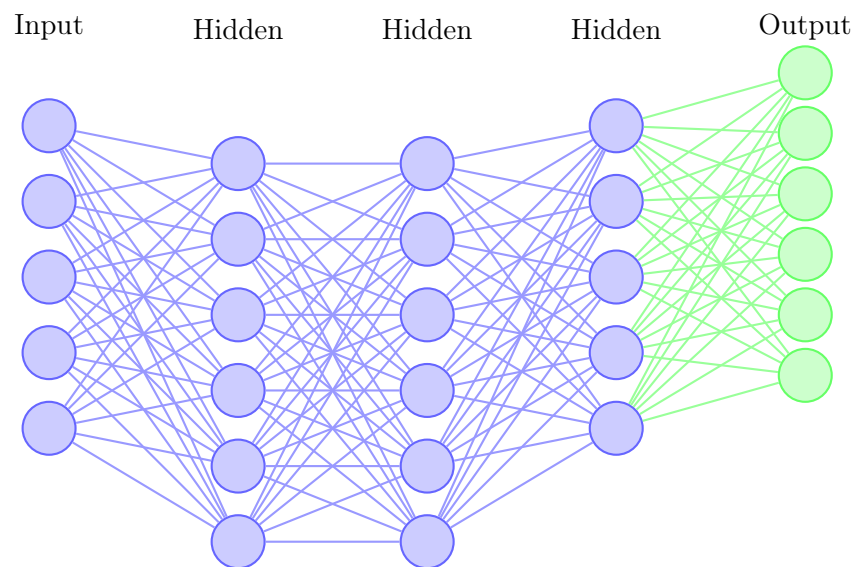


0 8 0 9 1 5

Wie erkennt
ein Computer
handgeschriebene Ziffern?



Inhaltsverzeichnis

Vorwort	3
1 Bilder, Computer und Bilderkennung	4
1.1 Was ist Bilderkennung?	4
1.2 Wie speichert ein Computer Bilder?	5
1.3 Der MNIST-Datensatz	7
2 Praktische Probleme	9
2.1 Das Problem der Handschrifterkennung	9
2.2 Ein erster Klassifikator: Durchschnittsbilder	12
2.3 Ein lernender Klassifikator: k-Nearest Neighbors (k-NN)	14
2.4 Warum verwenden wir dann neuronale Netze?	18
3 Künstliche Neuronen	19
3.1 Vom Vergleich zum Lernen: Das Perzeptron	19
3.2 Deep Learning: Viele Schichten lernen komplexe Muster	24
3.3 Convolutional Neural Networks (CNNs)	27
4 Experimente	31
4.1 Experiment 1: Einfluss der Aktivierungsfunktion	31
4.2 Experiment 2: Wie tief muss ein Netzwerk sein?	33
4.3 Experiment 3: Einfluss der Lernrate	34
4.4 Experiment 4: Dropout und Überanpassung	35
4.5 Experiment 5: Denses Netzwerk gegen CNN	36
4.6 Zusammenfassung der Experimente	37
5 Fazit	39
Anhang	41
5.1 Die Ableitung	41
5.2 Der Gradient	41
5.3 Lineare Entscheidungsgrenzen und die Rolle der Aktivierungsfunktion	42
5.4 Backpropagation in neuronalen Netzen	45
6 Literaturverzeichnis	48

Vorwort

Bei der klassischen Programmierung werden einem Computer sämtliche Verarbeitungsschritte explizit vorgegeben. Komplexe Problemstellungen werden dabei in viele kleine, klar definierte Teilaufgaben zerlegt, die der Computer schrittweise ausführt. Neuronale Netze verfolgen hingegen einen grundlegend anderen Ansatz. Anstatt den Lösungsweg vorzugeben, lernen sie selbstständig aus Beobachtungs- beziehungsweise Trainingsdaten und entwickeln eigenständig ein Modell zur Lösung der zugrunde liegenden Aufgabe.

Dieser datengetriebene Ansatz gilt als vielversprechend. Dennoch gelang es Wissenschaftlern bis zum Jahr 2006 – mit Ausnahme einiger spezialisierter Anwendungsfälle – nicht, neuronale Netze so zu trainieren, dass sie herkömmliche Verfahren zuverlässig übertrafen. Einen entscheidenden Fortschritt brachte die Entwicklung neuer Lernverfahren für sogenannte tiefe neuronale Netze. Diese Methoden werden heute unter dem Begriff *Deep Learning* zusammengefasst und seit ihrer Einführung kontinuierlich weiterentwickelt. Moderne Deep-Learning-Verfahren erzielen inzwischen herausragende Ergebnisse in zahlreichen Anwendungsbereichen, darunter die Bildverarbeitung, die Spracherkennung sowie die Verarbeitung natürlicher Sprache. Aufgrund ihrer hohen Leistungsfähigkeit werden sie heute in großem Umfang von Unternehmen und Forschungseinrichtungen eingesetzt.

Ziel dieses Kurses ist es, die grundlegenden Konzepte neuronaler Netze sowie moderne Deep-Learning-Verfahren zu vermitteln. Anhand des Problems der Erkennung handgeschriebener Ziffern werden die wichtigsten Methoden und Prinzipien des Deep Learnings schrittweise eingeführt und praktisch angewendet.

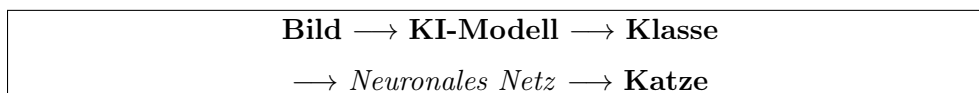
Kapitel 1

Bilder, Computer und Bilderkennung

1.1 Was ist Bilderkennung?

Bilderkennung¹ (auch *Image Recognition* oder *Computer Vision*) ist die Fähigkeit eines Computers, Bilder zu analysieren und zu verstehen. Ähnlich wie wir Menschen mit unseren Augen und unserem Gehirn erkennen Computer Muster, Formen, Farben und Objekte in Bildern.

Im Kern ist Bilderkennung häufig ein **Klassifikationsproblem**. Das bedeutet: Der Computer erhält ein Bild und soll entscheiden, zu welcher Kategorie es gehört.



Dabei kann es sich um einfache Klassen handeln, zum Beispiel:

- Katze oder Hund?
- Apfel oder Banane?
- Verkehrsschild oder Fußgänger?

Moderne KI-Systeme können jedoch nicht nur zwischen zwei Klassen unterscheiden, sondern aus Tausenden verschiedener Kategorien die wahrscheinlichste auswählen. Das Modell berechnet dabei für jede mögliche Klasse eine Wahrscheinlichkeit und wählt die Klasse mit der höchsten Wahrscheinlichkeit aus.

Damit ein Computer diese Aufgabe lösen kann, muss er zunächst aus vielen Beispielen lernen. Je mehr passende Trainingsdaten vorhanden sind, desto besser kann das Modell später unbekannte Bilder erkennen.

1.1.1 Beispiele aus dem Alltag

Bilderkennung begegnet uns heute in vielen Bereichen des täglichen Lebens:

- **Smartphone:** Das Telefon entsperrt sich durch Gesichtserkennung (z. B. Face ID).
- **Social Media:** Instagram oder Facebook können Personen auf Fotos automatisch erkennen und markieren.
- **Google Fotos:** Fotos werden automatisch nach Personen, Tieren oder Objekten sortiert und können später leicht gefunden werden.

¹Lernmaterialien <https://github.com/StefanoBelloni/Handschrifterkennung>

- **Autonome Fahrzeuge:** Kameras erkennen Fahrbahnmarkierungen, andere Fahrzeuge, Fußgänger und Verkehrsschilder.
- **Medizin:** KI unterstützt Ärztinnen und Ärzte dabei, Krankheiten auf Röntgen- oder MRT-Bildern frühzeitig zu erkennen.
- **Einzelhandel:** Kameras zählen Kundinnen und Kunden oder analysieren Laufwege im Geschäft.

Alle diese Anwendungen basieren auf derselben Grundidee: Eine Künstliche Intelligenz lernt aus vielen Beispielen, typische Muster in Bildern zu erkennen und diese anschließend einer passenden Klasse zuzuordnen.

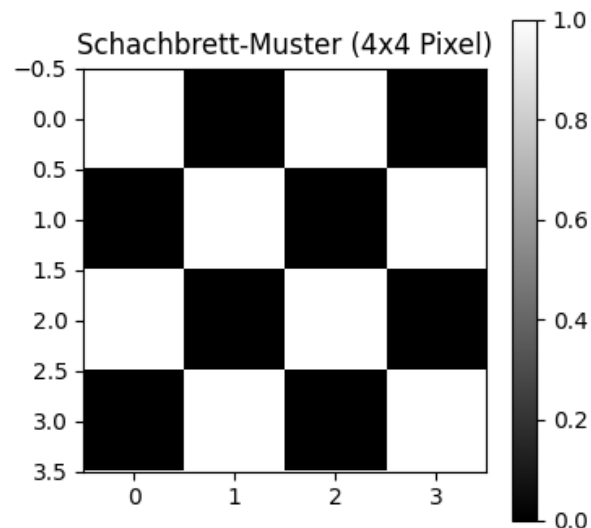
1.2 Wie speichert ein Computer Bilder?

Ein Bild auf dem Computer ist nichts anderes als eine **große Tabelle von Zahlen**, die man in der Mathematik als **Matrix** bezeichnet.

Ein Computer sieht also **keine Farben, keine Formen und keine Objekte**. Für ihn besteht ein Bild ausschließlich aus Zahlen.

1.2.1 Experiment 1: Ein einfaches Schwarzweiß-Bild

Stellen wir uns ein sehr kleines Bild mit einer Größe von nur 4×4 **Pixeln** vor. Ein **Pixel** (englisch: *Picture Element*) ist der kleinste Baustein eines digitalen Bildes.



Darstellung des Bildes

So speichert der Computer das Bild

$$\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Dabei bedeutet:

- 1 = schwarzes Pixel
- 0 = weißes Pixel

Für den Computer ist dieses Bild also lediglich eine Matrix aus Nullen und Einsen.

1.2.2 Experiment 2: Graustufenbilder

Schwarzweißbilder mit verschiedenen Helligkeiten werden als **Graustufenbilder** gespeichert.

Jedes Pixel erhält einen Wert zwischen 0 und 255:

- 0 = schwarz
- 128 = mittleres Grau
- 255 = weiß

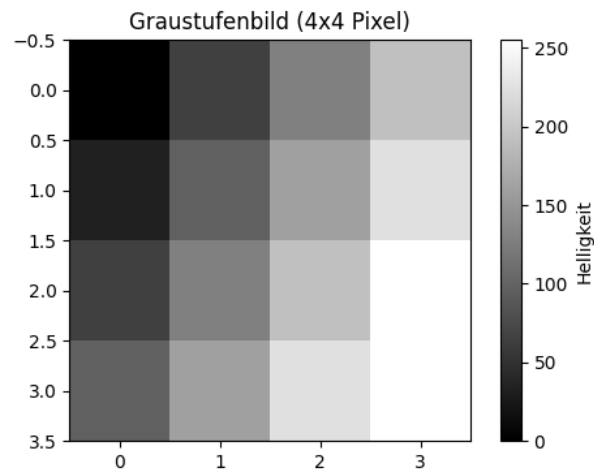
Ein mögliches 4×4 Bild könnte folgendermaßen aussehen:

$$\begin{bmatrix} 0 & 64 & 128 & 192 \\ 32 & 96 & 160 & 224 \\ 64 & 128 & 192 & 255 \\ 96 & 160 & 224 & 255 \end{bmatrix}$$

Je größer der Zahlenwert ist, desto heller erscheint das Pixel.

Mathematisch schreibt man:

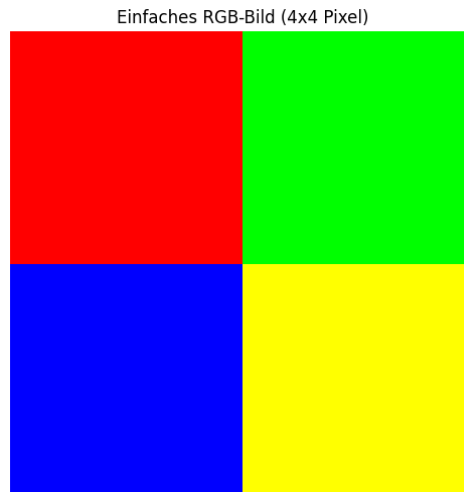
$$\text{Helligkeit} \in [0, 255].$$



1.2.3 Experiment 3: Farbbilder (RGB)

Farbbilder verwenden das **RGB-Farbmodell**. RGB steht für:

- Rot (Red)
- Grün (Green)
- Blau (Blue)



Jedes Pixel besitzt deshalb nicht nur einen, sondern **drei Zahlenwerte**.
Mathematisch:

$$\text{Pixel}(i, j) = (R_{i,j}, G_{i,j}, B_{i,j}), \quad R, G, B \in [0, 255].$$

Einige Beispiele:

Rot	(255, 0, 0)
Grün	(0, 255, 0)
Blau	(0, 0, 255)
Weiß	(255, 255, 255)
Schwarz	(0, 0, 0)

Man kann sich ein Farbbild als drei übereinanderliegende Matrizen vorstellen: eine für Rot, eine für Grün und eine für Blau.

1.2.4 Größere Bilder

Ein echtes Foto besteht aus sehr viel mehr Pixeln als die kleinen Beispiele zuvor.

Format	Auflösung	Anzahl Pixel	Zahlen pro Graustufenbild
HD	1920×1080	ca. 2,1 Mio.	ca. 2,1 Mio.
4K	3840×2160	ca. 8,3 Mio.	ca. 8,3 Mio.
12 MP Smartphone	ca. 4000×3000	12 Mio.	12 Mio.

Bei einem Farbbild werden für jedes Pixel drei Zahlen gespeichert (Rot, Grün und Blau). Ein Smartphone-Foto mit 12 Millionen Pixeln besteht daher aus insgesamt etwa **36 Millionen Zahlen**.

Merke:

Ein Computer verarbeitet Bilder nicht als Fotos, sondern als große Matrizen aus Zahlen. Genau diese Zahlen werden später von Verfahren der Künstlichen Intelligenz analysiert.

1.3 Der MNIST-Datensatz

In diesem Kurs arbeiten wir mit dem bekannten **MNIST-Datensatz**. Er enthält handgeschriebene Ziffern und wird seit vielen Jahren verwendet, um Verfahren der Bilderkennung und des Maschinellen Lernens zu testen.

Jedes Bild zeigt genau eine handgeschriebene Ziffer zwischen 0 und 9. Da die Ziffern von vielen verschiedenen Personen geschrieben wurden, sehen sie alle etwas unterschiedlich aus. Genau diese Unterschiede machen die Aufgabe für eine Künstliche Intelligenz interessant.

Eigenschaft	Wert
Anzahl Bilder	70 000
Trainingsbilder	60 000
Testbilder	10 000
Bildgröße	28×28 Pixel
Pixel pro Bild	784
Bildtyp	Graustufen (0–255)
Klassen	Ziffern 0 bis 9

1.3.1 Wie speichert der Computer ein MNIST-Bild?

Ein MNIST-Bild besitzt eine Größe von 28×28 Pixeln. Intern wird das Bild als Matrix gespeichert:

$$\underbrace{28 \times 28}_{=784 \text{ Pixel}}$$

Jedes Pixel besitzt einen Grauwert zwischen 0 (schwarz) und 255 (weiß).

Für viele Berechnungen wird diese Matrix anschließend zu einem langen Vektor mit insgesamt 784 Zahlen umgeformt:

$$\begin{bmatrix} 0 & 0 & \cdots & 255 \\ \vdots & \ddots & & \vdots \\ 30 & 120 & \cdots & 0 \end{bmatrix} \longrightarrow (0, 0, 0, 30, 100, \dots, 255).$$

Der Computer erkennt dabei keine Ziffer, sondern verarbeitet lediglich diese Zahlen.



Kapitel 2

Praktische Probleme

2.1 Das Problem der Handschrifterkennung

Stellen wir uns vor, wir ziehen zufällig eine handgeschriebene Ziffer aus einer Kiste und stellen die Frage:

„Welche Ziffer ist auf diesem Bild zu sehen?“

Für Menschen ist diese Aufgabe meist sehr einfach. Ein Computer muss dagegen lernen, wie er aus den Pixelwerten eines Bildes auf die richtige Ziffer schließen kann.

Die Aufgabe der Bilderkennung besteht also darin, jedem Bild genau eine der zehn Klassen

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

zuzuordnen¹.

2.1.1 Warum ist das schwierig?

Es gibt nicht die eine „richtige“ Schreibweise einer Ziffer. Jeder Mensch schreibt Zahlen etwas anders.

Unterschiede können zum Beispiel entstehen durch:

- unterschiedliche Handschriften,
- verschiedene Größen,
- leichte Drehungen oder Verschiebungen,
- unterschiedlich dicke Stifte,
- unterschiedlich stark geschriebene Linien.

Obwohl alle diese Bilder beispielsweise eine „7“ darstellen, unterscheiden sich ihre Pixelwerte teilweise deutlich. Für den Computer sehen sie daher zunächst wie völlig verschiedene Muster aus.

2.1.2 Naive Lösungsansätze

Bevor wir Künstliche Intelligenz verwenden, überlegen wir uns einige einfache Lösungsansätze.

¹Vorgeschlagene Literatur: [6], [1], insbesondere Kapitel 1 [5]

Idee 1: Feste Regeln aufstellen

Eine erste Idee wäre, jede Ziffer durch feste Merkmale zu beschreiben.

Beispielsweise könnte man sagen:

„Eine 7 besitzt oben eine waagerechte Linie und darunter eine schräge Linie.“

Auch dieser Ansatz hat Probleme:

- Manche Menschen schreiben eine 7 ohne waagerechte Linie.
- Andere Ziffern besitzen ähnliche Linien.
- Begriffe wie „oben“, „schräg“ oder „lang“ sind schwer mathematisch zu definieren.

Schon wenige Ausnahmen führen dazu, dass solche Regeln unzuverlässig werden.

Idee 2: Vergleich durch zusammengefasste Merkmale

Eine andere Idee wäre, ein Bild nicht Pixel für Pixel zu vergleichen, sondern einige wichtige Eigenschaften des Bildes zusammenzufassen.

Zum Beispiel könnten wir für jede Ziffer bestimmte Werte berechnen:

- die durchschnittliche Helligkeit aller Pixel,
- die Anzahl der dunklen Pixel,
- die Position des dunkelsten Bereichs,
- die Breite und Höhe der geschriebenen Ziffer,
- die Anzahl von Linien oder Kanten.

Damit würde ein Bild nicht mehr durch 784 einzelne Pixelwerte beschrieben, sondern durch eine kleine Anzahl von Merkmalen.

Beispielsweise könnte eine Ziffer so dargestellt werden:

$$\text{Bild} \longrightarrow \begin{pmatrix} \text{durchschnittliche Helligkeit} \\ \text{Anzahl dunkler Pixel} \\ \text{Breite} \\ \text{Höhe} \end{pmatrix}$$

Der Merkmalsraum Wenn wir nur zwei Merkmale verwenden, können wir jede Ziffer als einen Punkt in einem zweidimensionalen Raum darstellen.

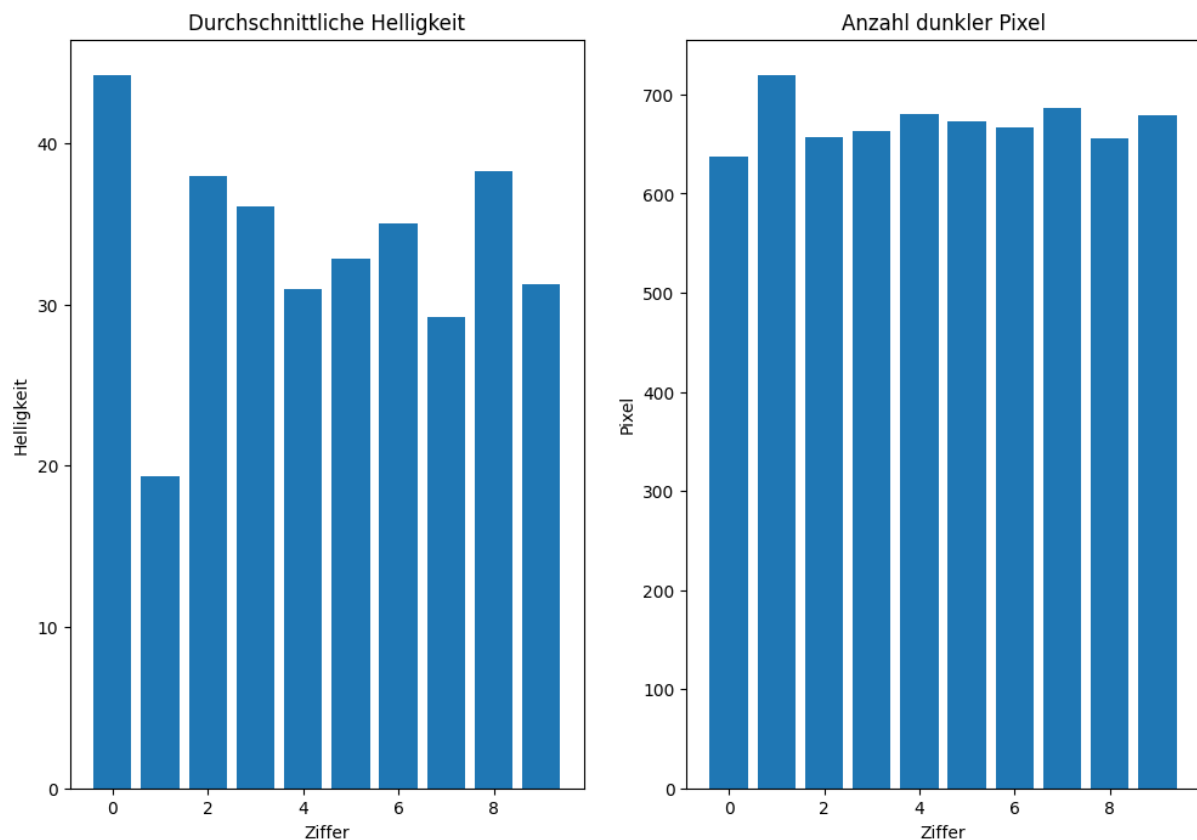
$$\text{Ziffer} \longrightarrow (\text{horizontaler Schwerpunkt}, \text{vertikaler Schwerpunkt})$$

Dadurch entsteht ein sogenannter **Merkmalsraum**.

Man erkennt jedoch auch die Grenzen dieser Methode: Einige Ziffern liegen sehr nah beieinander. Eine "3" und eine "8" können beispielsweise einen ähnlichen Schwerpunkt besitzen, obwohl ihre Formen völlig unterschiedlich sind.

Die Position eines Bildes im Merkmalsraum enthält also einige Informationen, aber nicht genug, um alle Ziffern zuverlässig zu erkennen.

Das Problem dabei ist jedoch: Diese Merkmale reichen nicht aus, um alle Ziffern zuverlässig zu unterscheiden.



Zwei verschiedene Ziffern können ähnliche Durchschnittswerte besitzen. Eine dick geschriebene "1" kann beispielsweise eine ähnliche Anzahl dunkler Pixel haben wie eine dünne "7". Auch die Position der Pixel allein beschreibt nicht die genaue Form einer Ziffer.

Außerdem ist es schwierig, die richtigen Merkmale von Hand auszuwählen:

- Welche Eigenschaften sind wirklich wichtig?
- Welche Merkmale unterscheiden eine "3" von einer "8"?
- Welche Merkmale funktionieren auch bei einer neuen Handschrift?

Die Form einer Ziffer besteht aus vielen kleinen Details und Zusammenhängen zwischen den Pixeln. Ein paar zusammengefasste Zahlenwerte können diese komplexen Muster nicht vollständig beschreiben.

2.1.3 Das eigentliche Problem

Die eigentliche Herausforderung lautet:

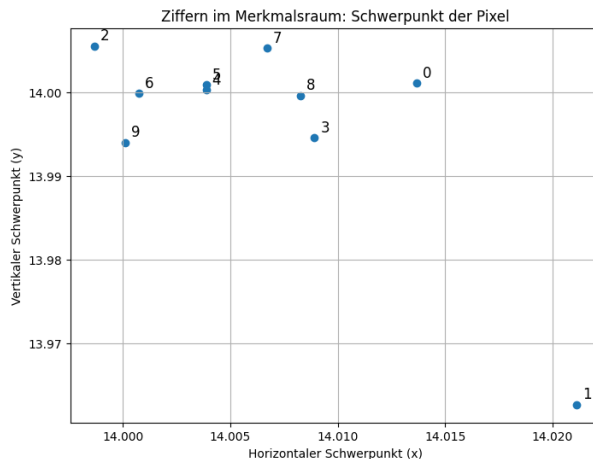
Wie findet man Regeln, die verschiedene Ziffern zuverlässig unterscheiden können?

Diese Regeln sind sehr kompliziert. Sie müssen gleichzeitig viele Eigenschaften eines Bildes berücksichtigen und trotzdem auch bei bisher unbekannten Handschriften funktionieren.

Menschen könnten solche Regeln kaum vollständig von Hand aufschreiben.

2.1.4 Die Idee des Maschinellen Lernens

Anstatt alle Regeln selbst zu formulieren, verfolgen wir einen anderen Ansatz:



Wir zeigen dem Computer viele bereits beschriftete Beispiele und lassen ihn die Regeln selbst lernen.

Der Computer erhält also tausende Bilder zusammen mit der richtigen Lösung:

Bild $\rightarrow$ richtige Ziffer

Aus diesen Beispielen erkennt das Modell selbst typische Muster und Zusammenhänge. Genau dieser Lernprozess bildet die Grundlage moderner Verfahren der Künstlichen Intelligenz und der Bilderkennung.

2.2 Ein erster Klassifikator: Durchschnittsbilder

Bevor neuronale Netze zur Klassifikation eingesetzt werden, wird zunächst ein sehr einfacher Klassifikator betrachtet.

Die Idee lautet:

Für jede Ziffer berechnen wir das durchschnittliche Bild aller Trainingsbeispiele.

Da der MNIST-Datensatz viele tausend Beispiele jeder Ziffer enthält, kann man für jede Pixelposition den Mittelwert berechnen.

Für die Ziffer 3 erhält man beispielsweise

$$\text{Durchschnittsbild} = \frac{1}{N} \sum_{i=1}^N \text{Bild}_i.$$

Das Ergebnis ist kein scharfes Bild mehr, sondern eine Art *typische* oder *durchschnittliche* Ziffer.

2.2.1 Die durchschnittlichen Ziffern

Im Notebook berechnen wir den Mittelwert aller Bilder jeder Klasse.

Dabei entsteht für jede Ziffer genau ein Prototyp.

Man erkennt, dass einige Ziffern sehr deutlich erscheinen, während andere etwas verschwommen wirken. Das liegt daran, dass verschiedene Menschen dieselbe Ziffer unterschiedlich schreiben.

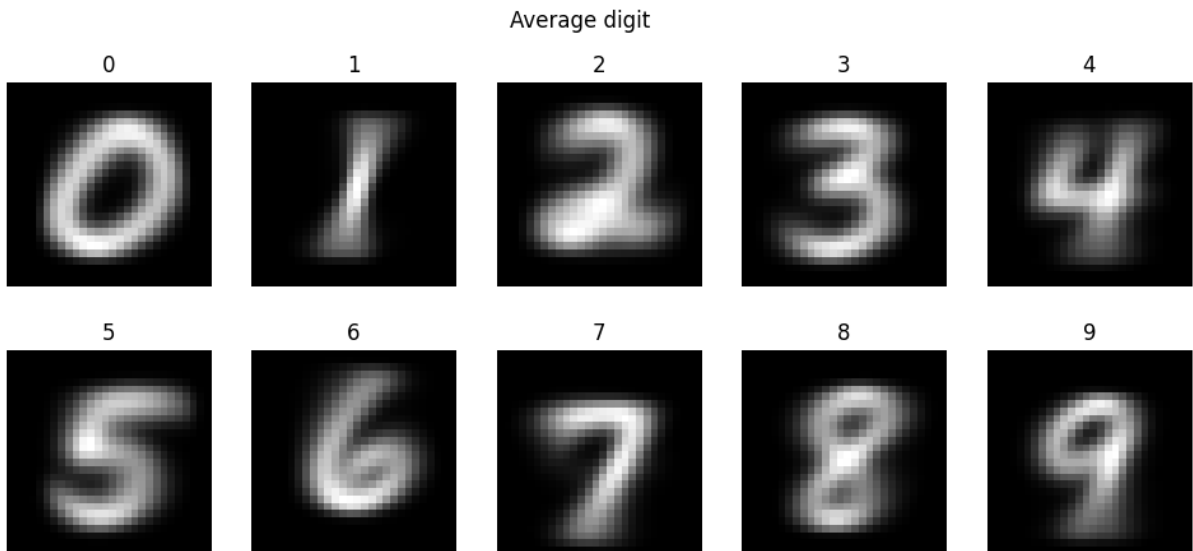


Abbildung 2.1: Die durchschnittlichen Ziffern des MNIST-Datensatzes.

2.2.2 Eine unbekannte Ziffer erkennen

Nun wird ein neues Bild mit jedem der zehn Durchschnittsbilder verglichen. Dazu wird für jede Ziffer der Abstand zwischen den Pixelwerten berechnet. Je kleiner dieser Abstand ist, desto ähnlicher sind sich die beiden Bilder.

Sei

$$x = (x_1, x_2, \dots, x_n)$$

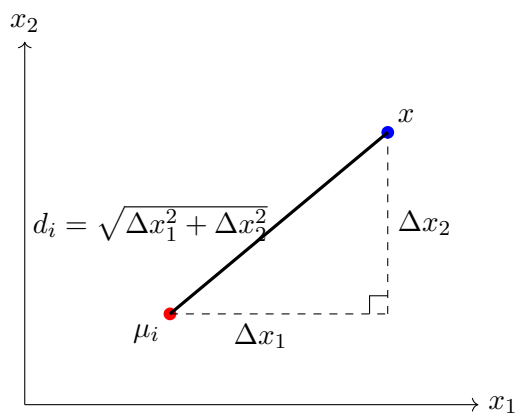
das neue Bild und

$$\mu_i = (\mu_{i,1}, \mu_{i,2}, \dots, \mu_{i,n})$$

das Durchschnittsbild der Ziffer i . Dabei entspricht jeder Eintrag einem Pixelwert und n der Gesamtzahl aller Pixel.

Als Ähnlichkeitsmaß wird die euklidische Distanz verwendet:

$$d_i = \|x - \mu_i\|_2 = \sqrt{\sum_{j=1}^n (x_j - \mu_{i,j})^2}.$$



Für jede mögliche Ziffer wird somit ein Abstand berechnet,

$$d_0, d_1, d_2, \dots, d_9,$$

wobei d_i den Abstand zwischen dem neuen Bild und dem Durchschnittsbild der Ziffer i beschreibt.

Die Klassifikation besteht nun darin, den kleinsten dieser Abstände zu finden. Mathematisch handelt es sich dabei um ein *Minimierungsproblem*. Gesucht ist diejenige Ziffer, deren Durchschnittsbild den geringsten Abstand zum Eingabebild besitzt.

Dies lässt sich kompakt schreiben als ²

$$\hat{y} = \arg \min_{i \in \{0, \dots, 9\}} d_i$$

Dieses Verfahren ist erstaunlich einfach und liefert bereits überraschend gute Ergebnisse. Figure 2.2 zeigt die *Konfusionsmatrix*³.

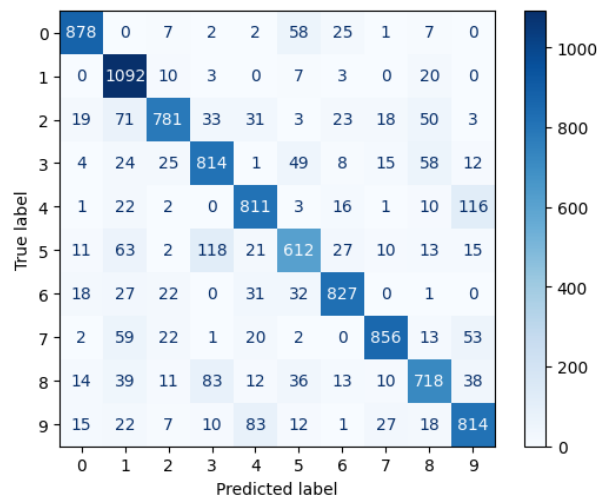


Abbildung 2.2: Konfusionsmatrix für Durchschnittsbilder Klassifikator

Dennoch besitzt es deutliche Grenzen. Handgeschriebene Ziffern unterscheiden sich häufig stark voneinander, sodass viele Beispiele nur wenig Ähnlichkeit mit ihrem jeweiligen Durchschnittsbild besitzen (Figure 2.3).

Dadurch entstehen Fehlklassifikationen, die mit leistungsfähigeren Verfahren, beispielsweise neuronalen Netzen, deutlich reduziert werden können.

2.3 Ein lernender Klassifikator: k-Nearest Neighbors (k-NN)

Unser erster Klassifikator hat für jede Ziffer genau ein Bild gespeichert: das Durchschnittsbild aller Trainingsdaten.

Dieses Verfahren ist sehr einfach und schnell. Allerdings geht dabei auch viel Information verloren. Jede Handschrift ist etwas anders und viele Besonderheiten verschwinden beim Bilden des Mittelwerts.

Wir stellen uns deshalb die Frage:

²Der Operator $\arg \min$ liefert dabei nicht den kleinsten Abstand selbst, sondern den Index, bei dem dieser minimale Abstand auftritt. Beispielsweise gilt

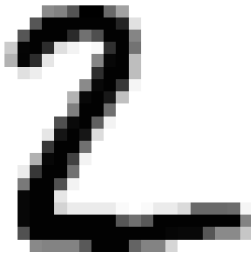
$$d_0 = 12, d_1 = 8, d_2 = 15, d_3 = 12, d_4 = 8, d_5 = 15, d_6 = 25, d_7 = 17, d_8 = 13, d_9 = 13,$$

Dann ist $\arg \min_i d_i = 1$,

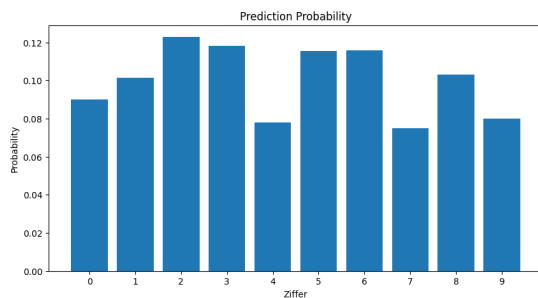
da der Abstand zur durchschnittlichen Ziffer 1 am kleinsten ist. Der Klassifikator entscheidet sich daher für die Ziffer 1.

³Eine *Confusion Matrix*, d.h. *Konfusionsmatrix* oder *Fehlermatrix* ist eine Tabelle, die die Leistung eines Klassifizierungsmodells beim maschinellen Lernen oder in der Statistik bewertet. Sie vergleicht die vom Modell vorhergesagten Werte mit den tatsächlichen Werten eines Datensatzes

Prediction: 2 True: 2

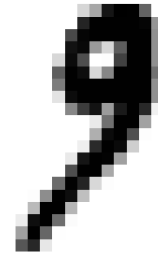


(a) Ziffer 2: predict 2

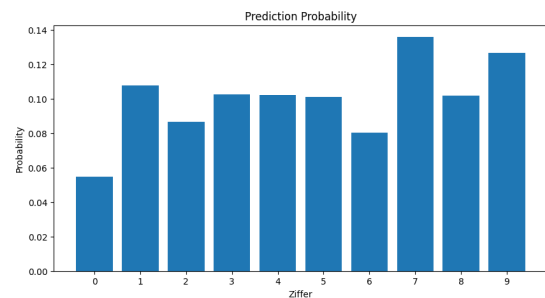


(c) Wahrscheinlichkeit Prediction

Prediction: 7 True: 9



(b) Ziffer 9: predict 7



(d) Wahrscheinlichkeit Prediction

Abbildung 2.3: Beispiele für Klassifikator Durchschnittsbilder

Warum speichern wir nicht einfach alle Trainingsbilder?

Genau diese Idee steckt hinter dem Verfahren **k-Nearest Neighbors (k-NN)**.

2.3.1 Die Grundidee

Anstatt nur zehn Durchschnittsbilder zu speichern, behalten wir alle Trainingsbilder.

Für den MNIST-Datensatz bedeutet das:

Durchschnittsbild-Klassifikator	k-NN
10 Bilder	60 000 Bilder

Der Computer besitzt dadurch viel mehr Beispiele und kann unbekannte Bilder besser mit bereits bekannten Handschriften vergleichen.

2.3.2 Der k-Nearest-Neighbors-Algorithmus

Angenommen, wir möchten eine unbekannte handgeschriebene Ziffer klassifizieren.

Der Computer führt nun vier einfache Schritte aus.

1. Berechne den Abstand zu *jedem* Trainingsbild.
2. Sortiere alle Trainingsbilder nach ihrem Abstand.
3. Wähle die k Bilder mit dem kleinsten Abstand.
4. Die *häufigste* Ziffer gewinnt.

Der Name **k-Nearest Neighbors** bedeutet also:

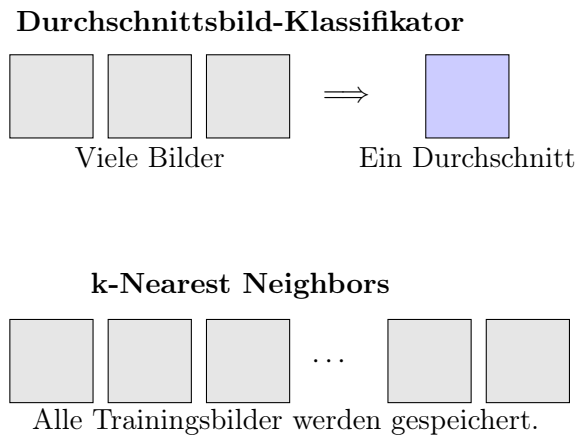


Abbildung 2.4: Der Durchschnittsbild-Klassifikator speichert nur einen Prototyp pro Klasse. Der k-NN-Klassifikator speichert dagegen alle Trainingsbilder.

- **Nearest** = die ähnlichsten Bilder
- **Neighbors** = die Nachbarn
- k = wie viele Nachbarn betrachtet werden

Ein typischer Wert ist beispielsweise $k = 5$.

Abstände berechnen Der wichtigste Schritt besteht darin, die Ähnlichkeit zweier Bilder zu berechnen.

Dazu betrachten wir jedes Pixel einzeln.

Sind zwei Bilder sehr ähnlich, dann unterscheiden sich ihre Pixelwerte nur wenig.

Mathematisch verwendet man häufig den sogenannten **euklidischen Abstand**

$$d(x, y) = \sqrt{\sum_{i=1}^{784} (x_i - y_i)^2}.$$

Dabei gilt:

- kleiner Abstand $\rightarrow$ Bilder sind sehr ähnlich
- großer Abstand $\rightarrow$ Bilder unterscheiden sich stark

Man erkennt sofort:

Die drei Bilder der Ziffer 7 besitzen die kleinsten Abstände.

Sie sehen dem Testbild also am ähnlichsten.

Die Mehrheitsentscheidung Nachdem alle Abstände berechnet wurden, betrachten wir nur die k nächsten Nachbarn.

Nehmen wir an, wir erhalten folgende Klassen:

$$\{8, \quad 8, \quad 3, \quad 8, \quad 5\}$$

Nun zählen wir einfach, welche Ziffer am häufigsten vorkommt.

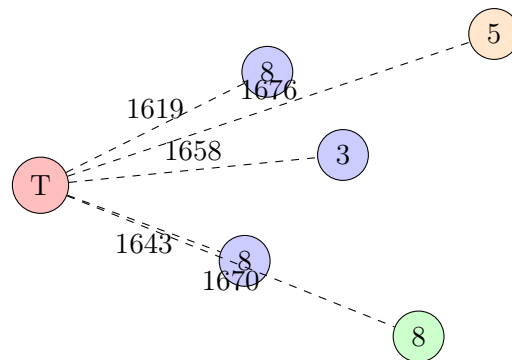


Abbildung 2.5: Der Abstand zwischen dem Testbild und allen Trainingsbildern wird berechnet. Kleine Abstände bedeuten hohe Ähnlichkeit.

Klasse	Anzahl
8	3
3	1
5	1

Da die Ziffer 8 am häufigsten vorkommt, lautet die Vorhersage **8**.

Dieses Verfahren nennt man eine **Mehrheitsentscheidung** (*Majority Vote*).

2.3.3 Eigenschaften des k-NN-Algorithmus

Die Grundidee des k-NN-Verfahrens lautet: *Ähnliche Bilder gehören meistens zur gleichen Klasse.*

Eine handgeschriebene 3 ähnelt normalerweise anderen handgeschriebenen 3en viel stärker als einer 8 oder einer 9.

Der Computer muss also keine komplizierten Regeln kennen.

Er nutzt lediglich die Erfahrung aus vielen bereits bekannten Beispielen (Figure 3.11).

Genau deshalb zählt k-NN bereits zu den Verfahren des **Maschinellen Lernens**.

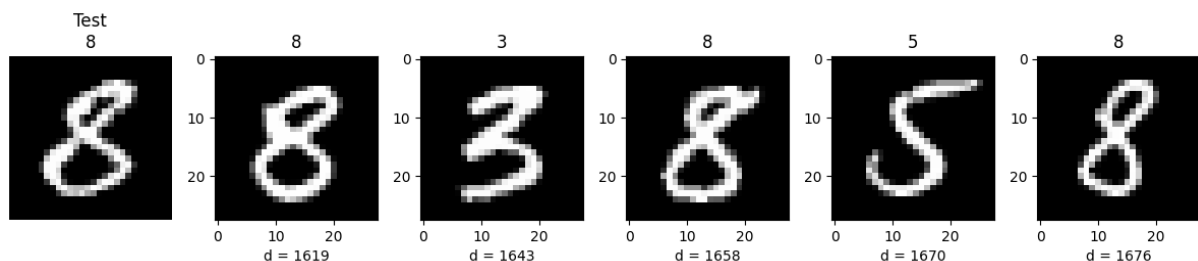


Abbildung 2.6: Beispiele Klassifizierung von Ziffer 8

Vorteile und Nachteile Wie jedes Verfahren besitzt auch k-NN Vor- und Nachteile.

Vorteile	Nachteile
Sehr einfach zu verstehen	Alle Trainingsbilder müssen gespeichert werden.
Hohe Genauigkeit	Für jedes neue Bild müssen alle 60 000 Trainingsbilder verglichen werden.
Keine komplizierten Regeln notwendig	Die Vorhersage dauert dadurch relativ lange.

Vergleich mit dem Durchschnittsbild-Klassifikator Der Durchschnittsbild-Klassifikator benötigt nur zehn Vergleiche.

Der k-NN-Klassifikator benötigt dagegen einen Vergleich mit jedem Trainingsbild.

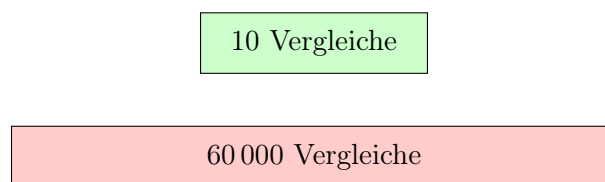


Abbildung 2.7: Vergleich des Rechenaufwands pro Vorhersage.

Der k-NN-Klassifikator ist also deutlich langsamer.

Dafür nutzt er alle vorhandenen Trainingsdaten und erreicht deshalb meist eine wesentlich höhere Genauigkeit.

2.4 Warum verwenden wir dann neuronale Netze?

Der Durchschnittsbild-Klassifikator ist sehr einfach aufgebaut und kann neue Bilder sehr schnell klassifizieren. Seine Genauigkeit ist jedoch begrenzt, da jede Ziffer lediglich durch ein einziges Durchschnittsbild repräsentiert wird. Individuelle Unterschiede in der Handschrift können dadurch nur unzureichend berücksichtigt werden.

Der *k-Nearest-Neighbors*-Klassifikator (k-NN) erzielt in der Regel eine deutlich höhere Genauigkeit. Anstatt ein Bild nur mit einem Durchschnittsbild zu vergleichen, werden die ähnlichsten Trainingsbilder gesucht. Die Vorhersage basiert anschließend auf den Klassen dieser Nachbarn. Der Nachteil dieses Verfahrens besteht darin, dass für jedes neue Bild der Abstand zu allen Trainingsbildern berechnet werden muss. Bei großen Datensätzen steigt dadurch der Rechenaufwand erheblich.

Neuronale Netze verfolgen einen anderen Ansatz. Während der Trainingsphase werden sämtliche Trainingsbilder verwendet, um eine mathematische Funktion zu erlernen, die Eingabebilder direkt auf die entsprechenden Klassen abbildet. Dabei werden die Parameter des Netzes so angepasst, dass die Vorhersagen auf den Trainingsdaten möglichst korrekt sind.

Nach Abschluss des Trainings werden die ursprünglichen Trainingsbilder für die Klassifikation neuer Bilder nicht mehr benötigt. Stattdessen genügt die gelernte Funktion, um aus den Pixelwerten eines neuen Bildes unmittelbar eine Vorhersage zu berechnen.

Neuronale Netze verbinden dadurch die Vorteile beider Verfahren. Wie der Durchschnittsbild-Klassifikator ermöglichen sie eine sehr schnelle Klassifikation neuer Bilder. Gleichzeitig erreichen sie eine Genauigkeit, die mit einfachen Verfahren wie dem Durchschnittsbild-Klassifikator nicht möglich ist und häufig sogar die Leistung des k-NN-Klassifikators übertrifft.

Kapitel 3

Künstliche Neuronen

3.1 Vom Vergleich zum Lernen: Das Perzeptron

Bisher haben wir verschiedene Möglichkeiten kennengelernt, Bilder zu klassifizieren:

- Beim Durchschnittsbild-Klassifikator *vergleichen* wir ein Bild mit einem Prototyp.
- Beim k-Nearest-Neighbors-Verfahren *vergleichen* wir ein Bild mit vielen Beispielen.

Beide Methoden verwenden jedoch direkt die vorhandenen Bilder.

Die nächste Idee ist grundlegend anders:

Kann ein Computer selbst eine mathematische Regel lernen, die Bilder voneinander unterscheidet?

Genau diese Idee bildet die Grundlage neuronaler Netze.

3.1.1 Das Perzeptron

Das Perzeptron ist eines der einfachsten künstlichen Neuronen¹.

Es erhält mehrere Eingaben und berechnet daraus eine Ausgabe.

Für ein MNIST-Bild bestehen die Eingaben aus den Pixelwerten:

$$x_1, x_2, \dots, x_{784}$$

Jedes Pixel erhält ein eigenes Gewicht:

$$w_1, w_2, \dots, w_{784}$$

Die Gewichte bestimmen, wie wichtig ein bestimmtes Pixel für die Entscheidung ist.

Mathematisch berechnet das Perzeptron zunächst eine gewichtete Summe:

$$z = w_1x_1 + w_2x_2 + \dots + w_{784}x_{784} + b$$

Kurz geschrieben:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Dabei bezeichnet:

¹Das Perzeptron wurde 1957 von Frank Rosenblatt entwickelt und 1958 veröffentlicht. Es war eines der ersten Modelle, das aus Beispieldaten lernen konnte, und gilt als Vorläufer moderner neuronaler Netze. Nachdem Marvin Minsky und Seymour Papert 1969 die Grenzen des Perzeptrons, insbesondere die Unfähigkeit zur Lösung nichtlinear separierbarer Probleme, aufgezeigt hatten, ging das Interesse an neuronalen Netzen zunächst zurück. Erst in den 1980er-Jahren und insbesondere mit dem Aufkommen des Deep Learnings ab 2006 erlebte dieses Forschungsgebiet einen erneuten Aufschwung.

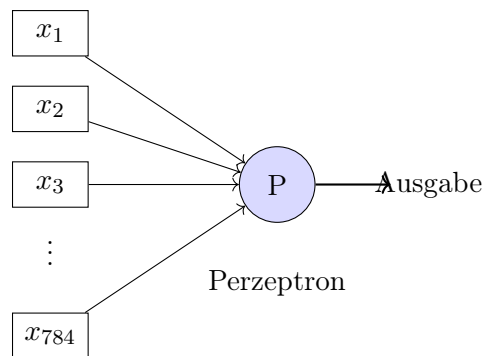


Abbildung 3.1: Ein Perzeptron kombiniert viele Eingaben zu einer Entscheidung.

- $\mathbf{x}$ die Pixelwerte des Bildes,
- $\mathbf{w}$ die gelernten Gewichte,
- b den Bias (eine zusätzliche Verschiebung).

Danach entscheidet eine Aktivierungsfunktion über die Ausgabe.
Eine einfache Variante wäre:

$$y = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Damit kann ein Perzeptron beispielsweise entscheiden:

Ist dieses Bild eine 0 oder keine 0?

3.1.2 Das eigentliche Problem: Welche Gewichte?

Die zentrale Frage lautet:

Wie finden wir die richtigen Gewichte $w_1, w_2, \dots, w_{784}$?

Am Anfang kennen wir diese Werte nicht.

Wir beginnen deshalb mit zufälligen Gewichten:

$$w_i \approx \text{zufälliger Wert}$$

Das Modell macht Vorhersagen, vergleicht diese mit der richtigen Antwort und verbessert anschließend die Gewichte.

Eine Verlustfunktion misst den Fehler Damit der Computer weiß, ob eine Änderung der Gewichte gut oder schlecht war, brauchen wir eine Fehlerfunktion.

Diese nennt man **Loss Function**.

Beispielsweise:

$$L(w) = (\text{Vorhersage} - \text{richtige Antwort})^2$$

Ein großer Fehler erzeugt einen großen Loss.

Das Ziel des Trainings ist *Minimiere den Fehler*

Die Suche nach dem Minimum Stellen wir uns vor, der Fehler hängt nur von einem Gewicht w ab.

Dann entsteht eine Kurve:

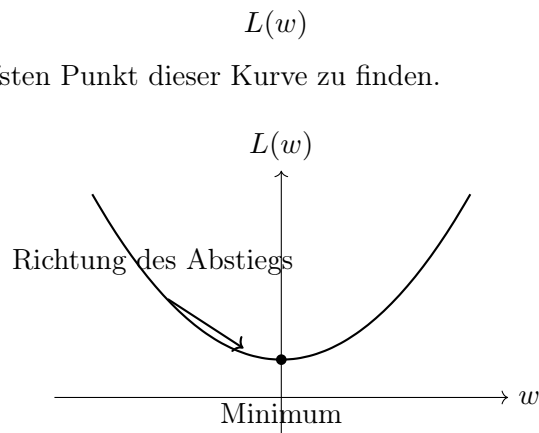


Abbildung 3.2: Die Verlustfunktion besitzt ein Minimum. Das Training sucht die Gewichte mit möglichst kleinem Fehler.

Der Gradient zeigt die Richtung Die Steigung der Kurve sagt uns, in welche Richtung wir gehen müssen. Diese Steigung nennt man **Gradient** (sehn Anhang 5.1 und 5.2).

Für ein einzelnes Gewicht:

$$\frac{\partial L}{\partial w}$$

bedeutet:

Wie verändert sich der Fehler, wenn wir das Gewicht ein kleines Stück verändern?

- Ist die Steigung positiv: $\left(\frac{\partial L}{\partial w} > 0\right) \Rightarrow$ müssen wir das Gewicht kleiner machen.
- Ist die Steigung negativ: $\left(\frac{\partial L}{\partial w} < 0\right) \Rightarrow$, dann müssen wir das Gewicht größer machen.

Der Lernschritt lautet:

$$w_{\text{neu}} = w_{\text{alt}} - \eta \frac{\partial L}{\partial w}$$

Der Parameter η heißt **Lernrate**.

Er bestimmt, wie groß die Schritte beim Lernen sind.

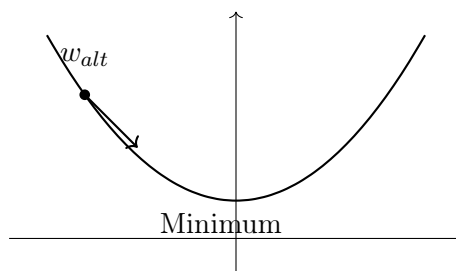


Abbildung 3.3: Der Gradient zeigt die Richtung, in der der Fehler kleiner wird.

3.1.3 Vom Perzeptron zum neuronalen Netz

Ein einzelnes Perzeptron kann nur sehr einfache Entscheidungen treffen.

Beispielsweise:

Ist dieses Bild eher eine 0 oder nicht?

Um komplexere Aufgaben zu lösen, verbinden wir viele Perzeptronen miteinander. Dadurch entstehen mehrere Schichten:

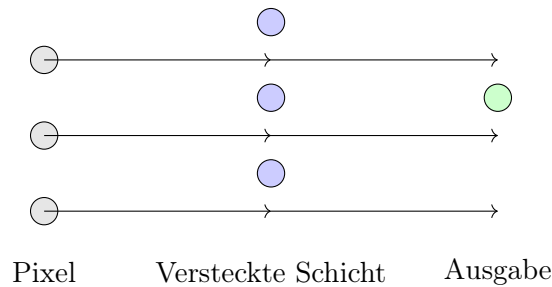


Abbildung 3.4: Ein neuronales Netz besteht aus vielen verbundenen Perzeptronen.

Ein modernes neuronales Netz lernt dadurch automatisch die wichtigen Merkmale eines Bildes:

Pixel $\rightarrow$ Linien $\rightarrow$ Formen $\rightarrow$ Ziffer

Genau diese Fähigkeit macht neuronale Netze so erfolgreich bei der Bilderkennung.

Für das Erlernen komplexer Muster sind nichtlineare Aktivierungsfunktionen in neuronalen Netzen von entscheidender Bedeutung.

3.1.4 Neuronale Netze und nichtlineare Aktivierungsfunktionen

Das Perzeptron besteht aus zwei Teilen:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

und einer Aktivierungsfunktion:

$$y = f(z)$$

Die Aktivierungsfunktion entscheidet, wie die Ausgabe des Neurons berechnet wird.

Das Problem einer rein linearen Aktivierung

Man könnte zunächst die Aktivierungsfunktion einfach weglassen:

$$y = z$$

Dann berechnet das Neuron nur eine lineare Funktion:

$$y = \mathbf{w} \cdot \mathbf{x} + b$$

Das scheint zunächst kein Problem zu sein.

Man könnte sogar mehrere Schichten miteinander verbinden:

Eingabe $\rightarrow$ Schicht 1 $\rightarrow$ Schicht 2 $\rightarrow$ Ausgabe

Aber mathematisch entsteht wieder nur eine einzige lineare Funktion.
Zum Beispiel:

$$y = W_2(W_1x + b_1) + b_2$$

kann umgeformt werden zu:

$$y = Wx + b$$

Die beiden Schichten bringen also keinen zusätzlichen Nutzen.
Egal wie viele lineare Schichten wir hinzufügen:

$$\text{linear} + \text{linear} + \text{linear} = \text{linear}$$

Das Netzwerk bleibt nur in der Lage, einfache gerade Grenzen zu lernen.

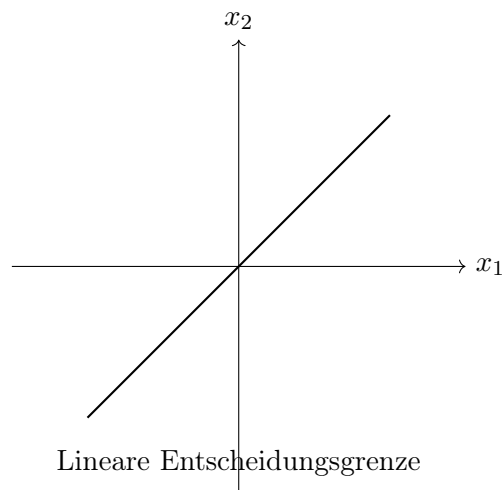


Abbildung 3.5: Ein Modell ohne Nichtlinearität kann nur gerade Trennlinien erzeugen.

Nichtlineare Aktivierungsfunktionen

Um komplexe Muster zu lernen, benötigen neuronale Netze nichtlineare Aktivierungsfunktionen.
Eine der ersten verwendeten Funktionen war die Sigmoid-Funktion:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sie verändert eine beliebige Zahl in einen Wert zwischen 0 und 1:

$$-\infty < x < \infty$$

wird zu

$$0 < \sigma(x) < 1$$

Heute wird häufig die ReLU-Funktion verwendet:

$$ReLU(x) = \max(0, x)$$

Sie ist sehr einfach:

$$ReLU(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}$$

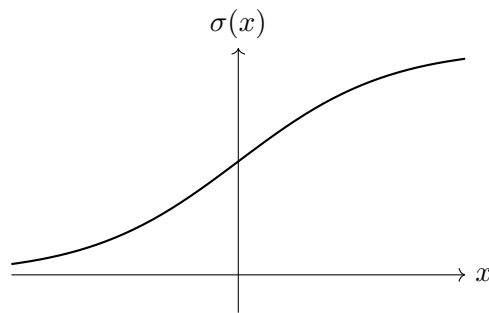


Abbildung 3.6: Die Sigmoid-Funktion erzeugt eine nichtlineare Transformation.

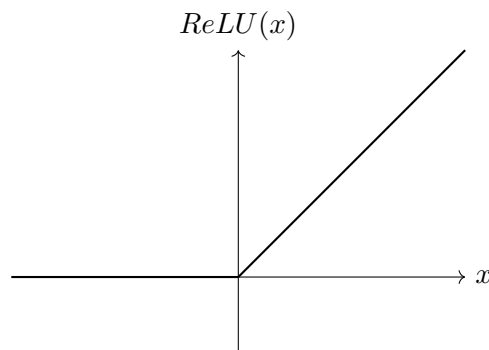


Abbildung 3.7: Die ReLU-Funktion ist für negative Werte null und für positive Werte linear.

Die Rolle von Nichtlinearitäten

Durch Aktivierungsfunktionen kann jede Schicht die Daten verändern.

Eine Schicht kann beispielsweise lernen:

Pixel $\rightarrow$ Kanten

Eine weitere Schicht:

Kanten $\rightarrow$ Formen

Eine weitere Schicht:

Formen $\rightarrow$ Ziffer

Ohne Nichtlinearitäten wären alle diese Schritte nur eine einzige große lineare Berechnung.

Mit Nichtlinearitäten kann ein neuronales Netz dagegen komplizierte Strukturen lernen. Für eine ausführlichere Diskussion sowie ein Beispiel siehe Anhang 5.3.

3.2 Deep Learning: Viele Schichten lernen komplexe Muster

Ein einzelnes Perzeptron kann nur sehr einfache Entscheidungen treffen.

Es kann beispielsweise lernen:

Ist ein Punkt links oder rechts von einer Linie?

Bei Bildern ist das Problem jedoch viel komplexer.

Eine handgeschriebene Ziffer besteht nicht aus einer einzigen Eigenschaft:

Pixel $\rightarrow$ Linien $\rightarrow$ Kurven $\rightarrow$ Formen $\rightarrow$ Ziffer

Die Idee von Deep Learning ist:

Statt dem Computer vorzugeben, welche Merkmale wichtig sind, lernt das Netzwerk diese Merkmale automatisch.

3.2.1 Von einem Neuron zum neuronalen Netzwerk

Ein künstliches Neuron berechnet:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

und anschließend:

$$a = f(z)$$

wobei f eine nichtlineare Aktivierungsfunktion ist.

Viele dieser Neuronen können miteinander verbunden werden.

Eine typische Struktur ist:

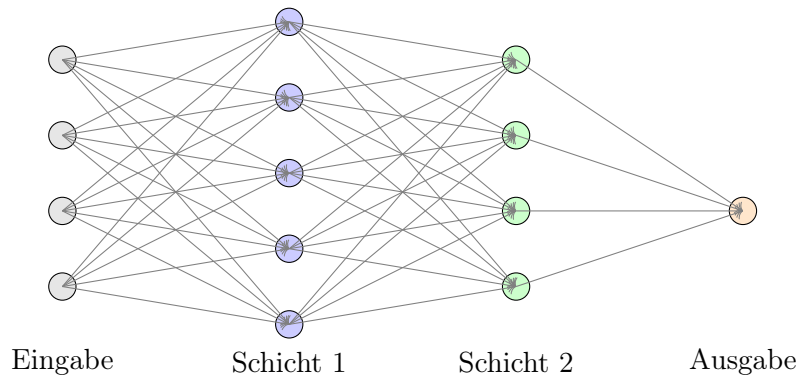


Abbildung 3.8: Ein künstliches neuronales Netzwerk besteht aus vielen verbundenen Neuronen.

Jede Verbindung besitzt ein eigenes Gewicht.

Ein Netzwerk mit mehreren Schichten besitzt deshalb sehr viele Parameter:

$$W_1, W_2, \dots, W_n$$

Beim Training werden diese Gewichte automatisch angepasst².

Der Lernprozess verwendet den Gradienten:

$$W_{\text{neu}} = W_{\text{alt}} - \eta \frac{\partial L}{\partial W}$$

Dieser Vorgang wird für alle Gewichte im Netzwerk wiederholt.

Das Wort **Deep** bedeutet nicht, dass das Netzwerk intelligent ist.

Es bedeutet lediglich:

$$\text{Deep Learning} = \text{Lernen mit vielen Schichten}$$

Ein Netzwerk mit:

$$\text{Eingabe} \rightarrow \text{eine Schicht} \rightarrow \text{Ausgabe}$$

²Backpropagation (Appendix 5.4). Dabei wird der Vorhersagefehler rückwärts durch das Netzwerk berechnet und mithilfe der Gradienteninformation werden die Gewichte so angepasst, dass der Fehler minimiert wird.

ist ein einfaches neuronales Netz.

Ein Netzwerk mit vielen Zwischenschichten:

$$\text{Eingabe} \rightarrow \underbrace{\text{Schicht} \rightarrow \text{Schicht} \rightarrow \dots \rightarrow \text{Schicht}}_{\text{viele Ebenen}} \rightarrow \text{Ausgabe}$$

wird Deep Neural Network genannt.

3.2.2 Lernen in tiefen neuronalen Netzen

Ein wichtiger Unterschied zu klassischen Verfahren ist:

Der Computer wählt die Merkmale selbst.

Bei einem traditionellen Ansatz müsste ein Mensch Merkmale definieren:

$$\text{Bild} \rightarrow \begin{cases} \text{Helligkeit} \\ \text{Kanten} \\ \text{Schwerpunkte} \\ \text{Formen} \end{cases} \rightarrow \text{Klassifikation}$$

Bei Deep Learning:

$$\text{Bild} \rightarrow \text{Neuronales Netzwerk} \rightarrow \text{Klassifikation}$$

Die ersten Schichten lernen häufig einfache Eigenschaften:

$$\text{Pixel} \rightarrow \text{Kanten}$$

Mittlere Schichten lernen komplexere Muster:

$$\text{Kanten} \rightarrow \text{Kurven und Formen}$$

Tiefe Schichten lernen abstrakte Konzepte:

$$\text{Formen} \rightarrow \text{Ziffer}$$

$$\text{Pixel} \longrightarrow \text{Kanten} \longrightarrow \text{Formen} \longrightarrow \text{Ziffer}$$

Abbildung 3.9: Tiefe Netzwerke lernen hierarchische Merkmale.

3.2.3 Die Bedeutung tiefer Netzwerke für die Bilderkennung

Bilder besitzen eine natürliche Struktur:

- benachbarte Pixel hängen zusammen,
- Linien bilden Formen,
- Formen bilden Objekte.

Ein tiefes Netzwerk kann diese Hierarchie automatisch lernen.
Bei der Handschrifterkennung:

$$28 \times 28 = 784$$

Pixel werden als Eingabe verwendet.
Das Netzwerk lernt selbst:

$$784 \text{ Pixel} \rightarrow \text{Merkmale} \rightarrow 10 \text{ Klassen}$$

Die Ausgabe besteht aus zehn Wahrscheinlichkeiten:

$$P(0), P(1), \dots, P(9)$$

Beispiel:

Ziffer	Wahrscheinlichkeit
0	0.01
1	0.02
2	0.03
3	0.85
4	0.01
...	

Das Modell entscheidet:

höchste Wahrscheinlichkeit = Vorhersage

3.3 Convolutional Neural Networks (CNNs)

Ein normales neuronales Netzwerk (Dense Network) behandelt ein Bild als eine lange Liste von Zahlen:

$$28 \times 28 = 784$$

Pixel werden einfach zu einem Vektor umgeformt:

$$(784) \rightarrow (128) \rightarrow (64) \rightarrow (10)$$

Das Netzwerk weiß dabei nicht, dass bestimmte Pixel nebeneinander liegen.
Für ein Bild ist diese Information aber sehr wichtig:

- benachbarte Pixel bilden Linien,
- Linien bilden Formen,
- Formen bilden Objekte.

Ein CNN (Convolutional Neural Network) wurde speziell entwickelt, um diese *räumliche Struktur* von Bildern auszunutzen.

3.3.1 Die Grundidee einer Faltung

Eine Faltung (*Convolution*) untersucht kleine lokale Bereiche eines Bildes. Im Gegensatz zu einem vollständig verbundenen Netzwerk wird nicht das gesamte Bild gleichzeitig betrachtet. Stattdessen wird ein kleines Fenster schrittweise über das Bild bewegt.

Dieses Fenster wird als **Filter** oder **Kernel** bezeichnet. Ein Kernel enthält Gewichte, die während des Trainings eines neuronalen Netzes gelernt werden. Ein einfaches Beispiel ist ein 3×3 -Kernel:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Dieser Filter berechnet für jeden Bildausschnitt eine gewichtete Summe der Pixelwerte. Dazu wird der Kernel auf einen kleinen Bereich des Bildes gelegt. Jeder Pixelwert wird mit dem entsprechenden Kernelgewicht multipliziert und anschließend werden alle Ergebnisse addiert:

$$s = \sum_{i=1}^3 \sum_{j=1}^3 x_{ij} K_{ij}.$$

Der berechnete Wert s beschreibt, wie stark das betrachtete Bildsegment mit dem Filter übereinstimmt. Wird der Kernel über das gesamte Bild verschoben, entsteht eine neue zweidimensionale Darstellung, die sogenannte **Feature Map**:

$$\text{Feature Map} = X * K.$$

Bei einem Kantenfilter entstehen beispielsweise hohe Werte an Stellen, an denen eine vertikale Helligkeitsänderung auftritt. Der oben gezeigte Kernel vergleicht dabei die linke und rechte Seite eines Bildausschnitts und kann dadurch vertikale Kanten hervorheben.

Der entscheidende Vorteil einer Faltung besteht darin, dass nicht jedes Pixel mit jedem anderen Pixel verbunden werden muss. Der Filter betrachtet nur lokale Strukturen und wird für das gesamte Bild wiederverwendet. Dadurch benötigt ein Convolutional Neural Network deutlich weniger Parameter als ein vollständig verbundenes Netzwerk und kann gleichzeitig wichtige Eigenschaften von Bildern, wie Kanten, Formen und Texturen, erkennen.

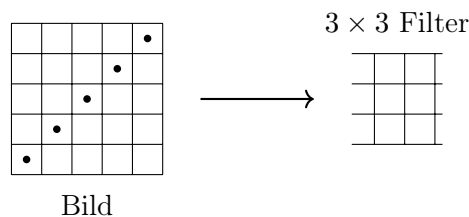


Abbildung 3.10: Ein Filter untersucht kleine Bereiche eines Bildes.

Für jeden Bildausschnitt wird berechnet:

$$\text{Ausgabe} = \sum_{i,j} \text{Pixel}_{i,j} \cdot \text{Filter}_{i,j}$$

Der Filter besitzt eigene Gewichte.

Diese Gewichte werden genauso wie bei einem normalen neuronalen Netzwerk durch Training gelernt.

3.3.2 Feature Maps

Wenn ein Filter über ein Bild bewegt wird, entsteht eine neue Darstellung:

Bild $\rightarrow$ Feature Map

Eine Feature Map zeigt, wo ein bestimmtes Merkmal gefunden wurde.
Beispiele:

- ein Filter erkennt horizontale Linien,
- ein anderer Filter erkennt Kreise,
- ein anderer Filter erkennt Ecken.

Ein CNN verwendet viele Filter gleichzeitig:

$$\text{Bild} \rightarrow \begin{bmatrix} \text{Kanten} \\ \text{Kurven} \\ \text{Strukturen} \end{bmatrix}$$

3.3.3 Die Vorteile von CNNs für die Bildverarbeitung

Problem bei Dense Netzwerken

Ein Dense Netzwerk verbindet jedes Eingabepixel mit jedem Neuron:

$$784 \times 128 = 100352$$

Gewichte werden bereits für die erste Schicht benötigt.

Außerdem verliert das Netzwerk die Information über die Position:

$$\begin{bmatrix} A & B & C \\ D & E & F \\ G & H & I \end{bmatrix}$$

und

$$\begin{bmatrix} I & H & G \\ F & E & D \\ C & B & A \end{bmatrix}$$

enthalten dieselben Zahlen, aber eine völlig andere Struktur.

Vorteile von CNNs

CNNs verwenden lokale Verbindungen: 3×3 statt 28×28

Dadurch entstehen mehrere Vorteile:

- **Weniger Parameter:** Ein Filter mit Größe: 3×3 benötigt nur 9 Gewichte. Diese Gewichte werden über das gesamte Bild wiederverwendet.
- **Erkennen von Positionen** Ein Filter für eine Kante funktioniert egal, ob die Kante links oder rechts im Bild erscheint. Das nennt man **Translation Invariance**
- **Hierarchisches Lernen** Mehrere CNN-Schichten lernen immer komplexere Eigenschaften:

Schicht 1 : Kanten

Schicht 2 : Kurven und einfache Formen

Schicht 3 : Ziffern und Objekte

3.3.4 Pooling: Wichtige Informationen behalten

Nach einer Faltung wird häufig eine Pooling-Schicht verwendet.

Beispiel:

$$2 \times 2$$

Max-Pooling:

$$\begin{bmatrix} 1 & 5 \\ 2 & 3 \end{bmatrix} \rightarrow 5$$

Die Idee:

- kleine Details werden entfernt,
- wichtige Merkmale bleiben erhalten,
- das Bild wird kleiner.

Dadurch wird das Netzwerk robuster gegenüber kleinen Verschiebungen.

3.3.5 Aufbau eines CNN für MNIST

Ein typisches CNN für die Handschrifterkennung:

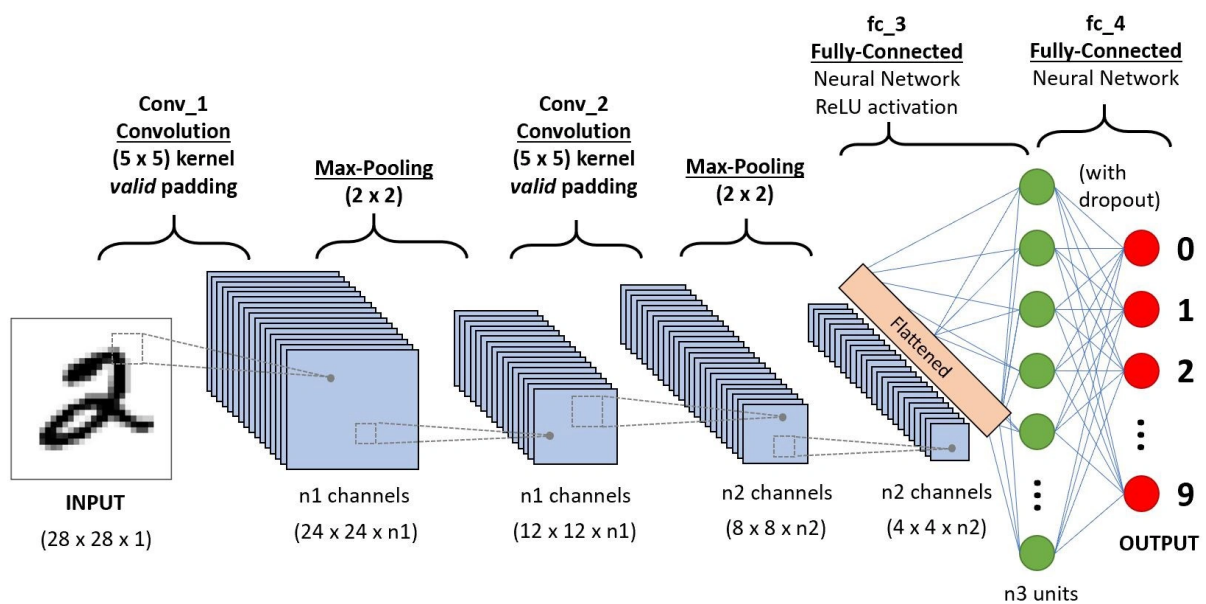


Abbildung 3.11: Ein typisches CNN für die Handschrifterkennung

<https://www.analyticsvidhya.com/blog/2020/10/what-is-the-convolutional-neural-network-architecture/>

Das Modell entscheidet für dir Klasse mit größter Wahrscheinlichkeit

Kapitel 4

Experimente

In diesem Kapitel untersuchen wir verschiedene Eigenschaften neuronaler Netze praktisch.

Wir verändern jeweils nur einen Teil des Modells und beobachten, wie sich das Verhalten verändert.

Dabei betrachten wir nicht nur die Genauigkeit, sondern auch:

- Trainingszeit,
- Anzahl der Parameter,
- Lernverlauf,
- Fehler bei der Klassifikation.

Das Ziel ist nicht nur ein möglichst gutes Modell zu bauen, sondern zu verstehen, warum bestimmte Architekturen besser funktionieren.

4.1 Experiment 1: Einfluss der Aktivierungsfunktion

4.1.1 Fragestellung

Welche Aktivierungsfunktion eignet sich am besten für ein neuronales Netz?

Wir vergleichen:

- ReLU,
- Sigmoid,
- Tanh.

4.1.2 Hintergrund

Ein Neuron berechnet zunächst:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

und anschließend:

$$a = f(z)$$

Die Aktivierungsfunktion bringt die notwendige Nichtlinearität in das Netzwerk. Ohne Aktivierungsfunktion wäre:

$$\textit{Linear} + \textit{Linear} + \textit{Linear} = \textit{Linear}$$

Das Netzwerk könnte dann keine komplexen Muster lernen.

4.1.3 Experiment

Wir verwenden dieselbe Architektur:

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

Nur die Aktivierungsfunktion wird verändert.

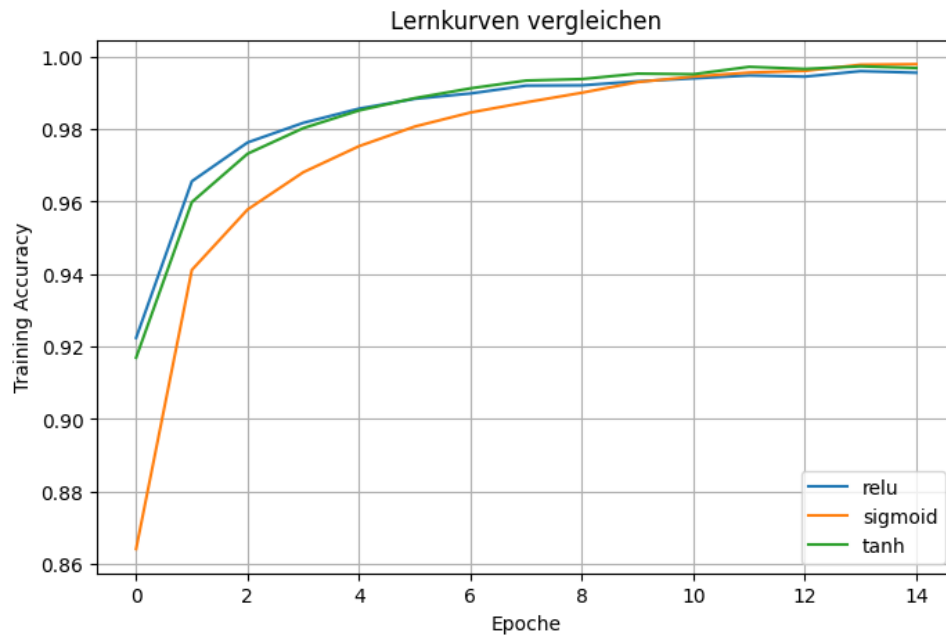


Abbildung 4.1: Vergleich Lernkurve verschiedener Aktivierungsfunktionen.

4.1.4 Ergebnisse

Die Modelle werden anhand folgender Werte verglichen:

- Testgenauigkeit,
- Trainingszeit,
- Lernkurve.

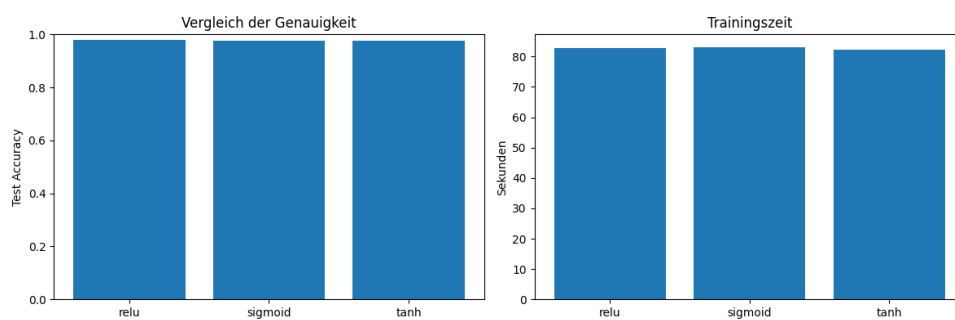


Abbildung 4.2: Vergleich der Genauigkeit und Trainingszeit verschiedener Aktivierungsfunktionen.

4.1.5 Interpretation

Typischerweise funktioniert ReLU besser als Sigmoid.

Der Grund:

- ReLU erzeugt stärkere Gradienten,
- Training mit vielen Schichten ist stabiler,
- Berechnung ist einfacher.

4.2 Experiment 2: Wie tief muss ein Netzwerk sein?

4.2.1 Fragestellung

Verbessert eine größere Anzahl von Schichten automatisch das Ergebnis?

4.2.2 Hypothese

Mehr Schichten können komplexere Muster lernen.

Allerdings können sehr große Netzwerke auch Probleme verursachen:

- längere Trainingszeit,
- mehr Parameter,
- Überanpassung.

4.2.3 Experiment

Wir vergleichen verschiedene Architekturen:

Modell A: $784 \rightarrow 32 \rightarrow 10$

Modell B: $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$

Modell C: $784 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 10$

4.2.4 Ergebnisse

4.2.5 Interpretation

Ein größeres Netzwerk besitzt mehr lernbare Parameter:

mehr Parameter $\rightarrow$ mehr Modellkapazität

Dies bedeutet aber nicht automatisch bessere Ergebnisse.

Das Netzwerk muss ausreichend Daten sehen und richtig trainiert werden.

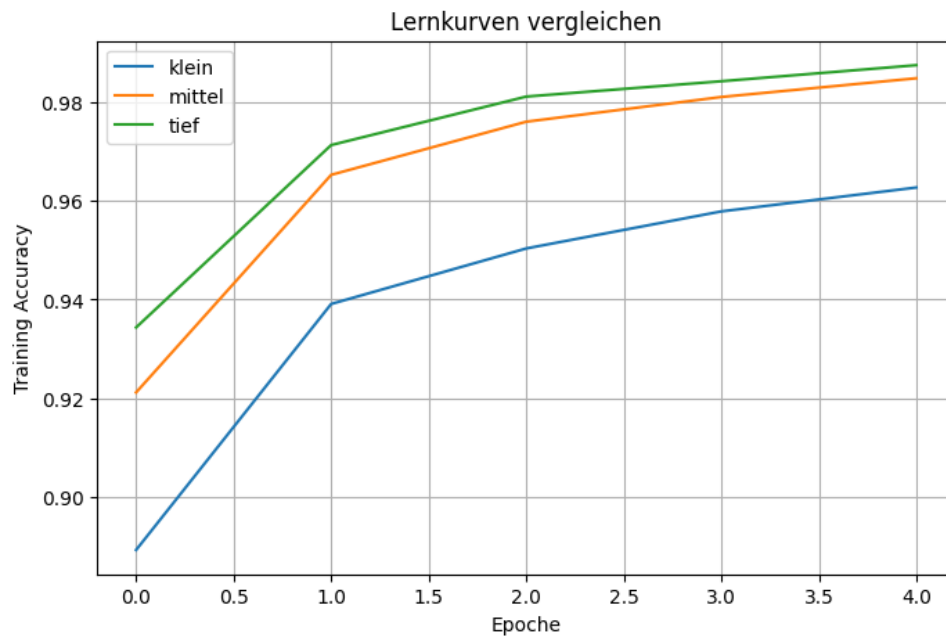


Abbildung 4.3: Lernkurve für Verschiedene Netzwerkgrößen.

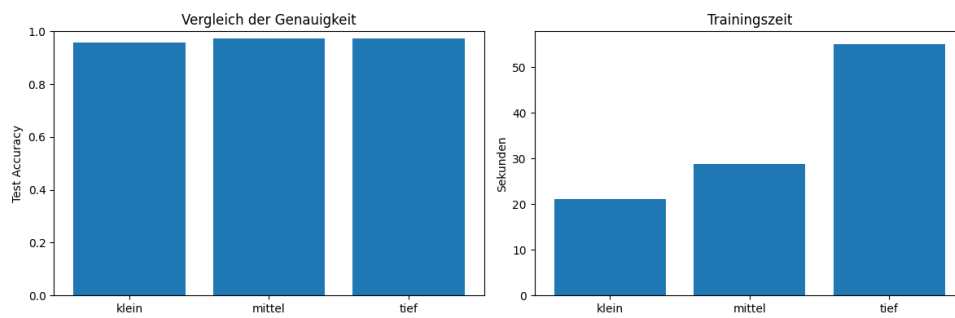


Abbildung 4.4: Einfluss der Netzwerkgröße auf die Genauigkeit und Trainingszeit.

4.3 Experiment 3: Einfluss der Lernrate

4.3.1 Fragestellung

Wie groß sollten die Schritte beim Lernen sein?

Beim Gradient Descent werden die Gewichte angepasst:

$$W_{neu} = W_{alt} - \eta \frac{\partial L}{\partial W}$$

Der Parameter η ist die Lernrate.

4.3.2 Experiment

Wir vergleichen:

$$\eta \in \{0.0001, 0.001, 0.01, 0.1\}$$

4.3.3 Interpretation

Eine zu kleine Lernrate:

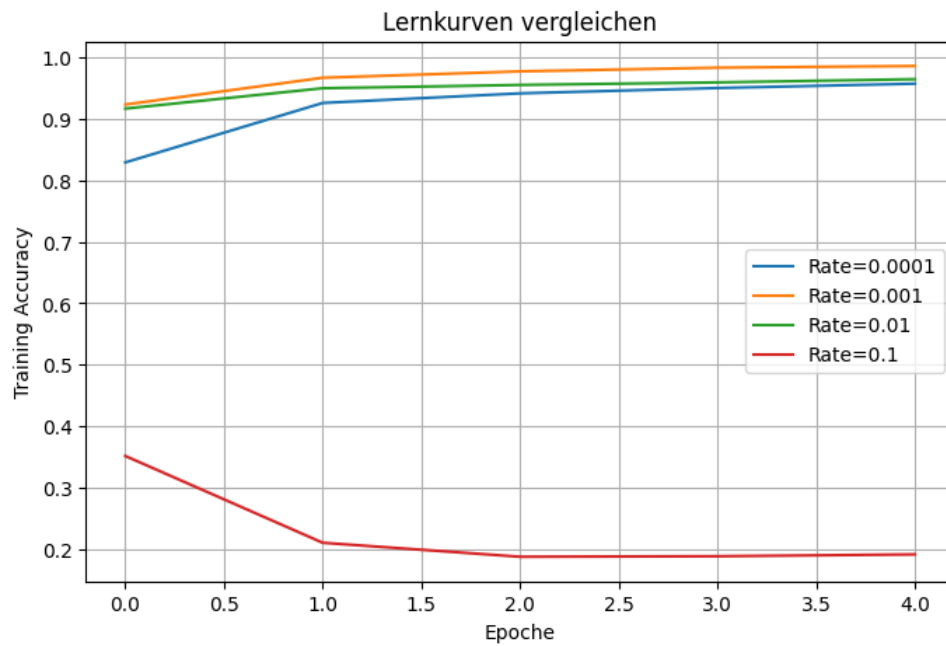


Abbildung 4.5: Training mit verschiedenen Lernraten.

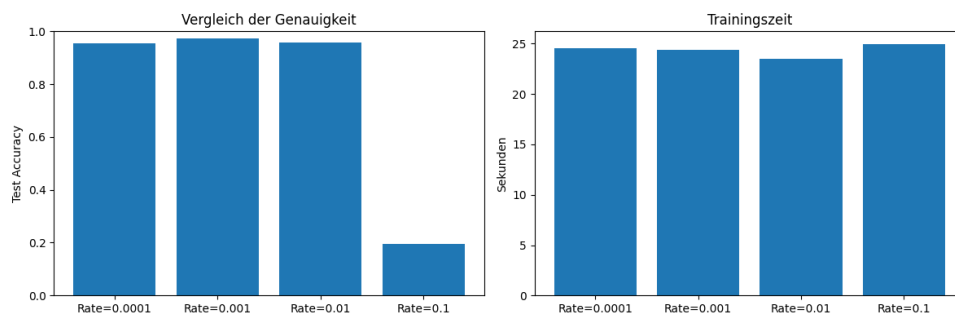


Abbildung 4.6: Vergleich der Genauigkeit und Trainingszeit verschiedener Lernraten.

→ Training sehr langsam

Eine zu große Lernrate:

→ Minimum wird übersprungen

Eine gute Lernrate ermöglicht schnelles und stabiles Lernen.

4.4 Experiment 4: Dropout und Überanpassung

4.4.1 Fragestellung

Kann ein Netzwerk zu viel auswendig lernen?

Ein Modell soll nicht nur die Trainingsdaten erkennen:

Training $\neq$ Realität

Es soll auf neue Bilder generalisieren.

4.4.2 Idee von Dropout

Während des Trainings werden zufällig einzelne Neuronen deaktiviert.

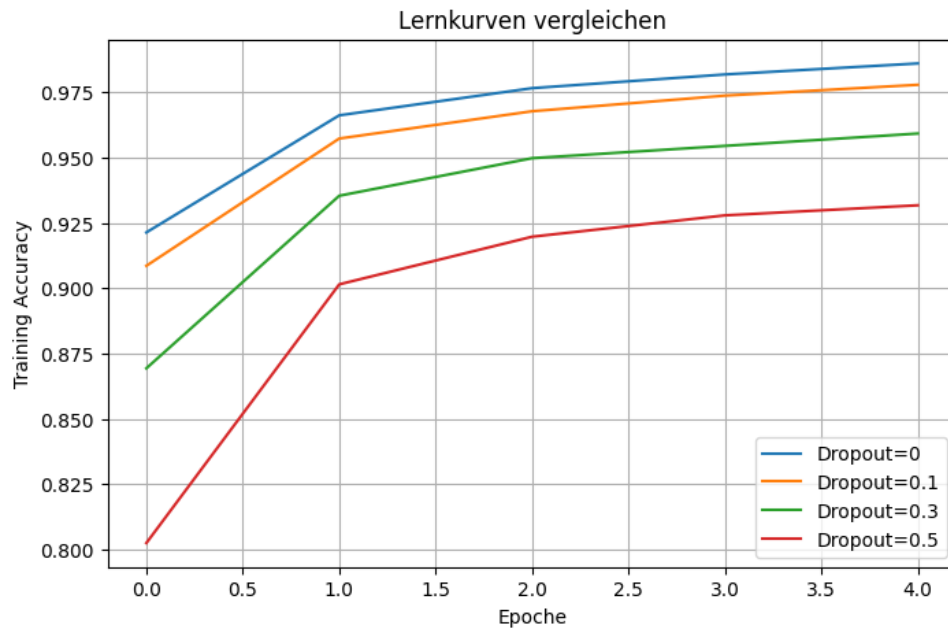


Abbildung 4.7: Dropout entfernt zufällig Verbindungen während des Trainings.

Dadurch lernt das Netzwerk robustere Eigenschaften.

4.4.3 Experiment

Vergleich:

$$Dropout \in \{0, 0.1, 0.3, 0.5\}$$

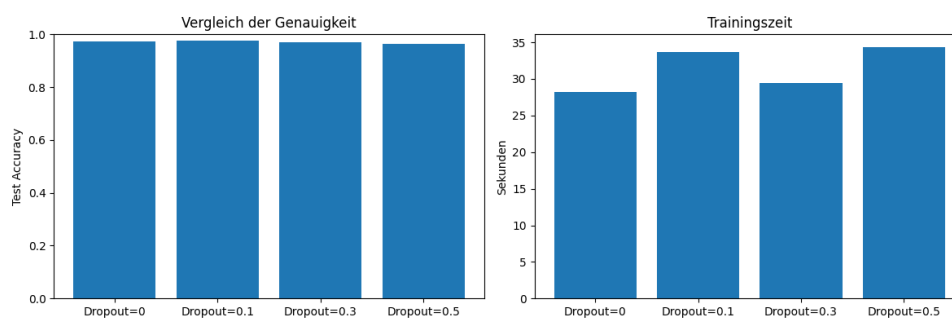


Abbildung 4.8: Einfluss von Dropout auf die Generalisierung.

4.4.4 Interpretation

Dropout kann die Trainingsgenauigkeit reduzieren, aber die Testgenauigkeit verbessern.

4.5 Experiment 5: Dense Netzwerk gegen CNN

4.5.1 Fragestellung

Warum verwenden moderne Bilderkennungssysteme CNNs?

4.5.2 Vergleich

Dense Netzwerk:

$$784 \rightarrow 128 \rightarrow 64 \rightarrow 10$$

CNN:

$$28 \times 28 \rightarrow \text{Conv} \rightarrow \text{Pooling} \rightarrow \text{Dense} \rightarrow 10$$

Abbildung 4.9: Dense Netzwerk und CNN im Vergleich.

4.5.3 Ergebnisse

Wir vergleichen:

- Genauigkeit,
- Anzahl Parameter,
- Fehlerbilder.

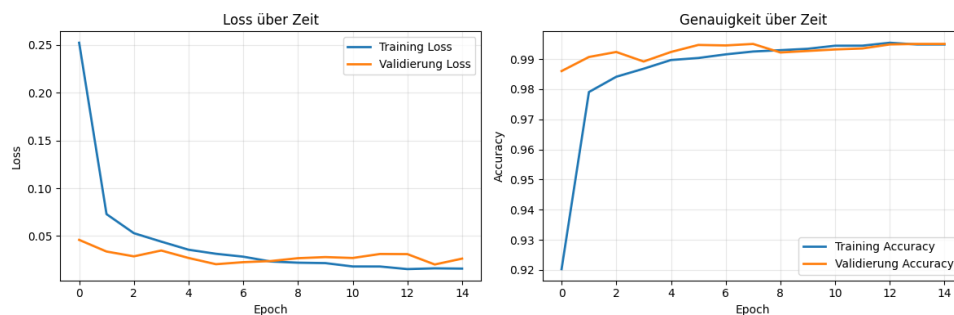


Abbildung 4.10: Loss und Genauigkeit über Zeit

4.5.4 Interpretation

CNNs nutzen die Struktur von Bildern:

$$\text{Pixel} \rightarrow \text{Kanten} \rightarrow \text{Formen} \rightarrow \text{Objekte}$$

Dadurch können sie Bildinformationen effizienter verarbeiten.

4.6 Zusammenfassung der Experimente

Die Experimente zeigen:

- Aktivierungsfunktionen ermöglichen Nichtlinearität.
- Mehr Schichten erhöhen die Modellkapazität.
- Die Lernrate bestimmt die Geschwindigkeit des Trainings.
- Dropout verbessert die Generalisierung.

- CNNs sind für Bilder besser geeignet als normale Dense Netzwerke.

Gute künstliche Intelligenz entsteht nicht nur durch ein großes Modell, sondern durch die passende Kombination aus Daten, Architektur und Training.

Kapitel 5

Fazit

In diesem Kurs haben wir den grundlegenden Weg von der klassischen Bilderkennung bis zu modernen neuronalen Netzen kennengelernt.

Ausgangspunkt war die Frage:

Wie kann ein Computer handgeschriebene Ziffern erkennen?

Zunächst haben wir gesehen, dass ein Computer Bilder nicht als Formen oder Objekte wahrnimmt. Ein digitales Bild besteht lediglich aus einer Matrix von Zahlen, wobei jeder Eintrag die Helligkeit eines Pixels beschreibt.

Anschließend haben wir verschiedene Möglichkeiten betrachtet, Bilder zu klassifizieren.

Einfache Ansätze, wie der Vergleich einzelner Kennzahlen (z. B. der durchschnittlichen Helligkeit) oder das Speichern aller bekannten Beispiele, führen schnell an ihre Grenzen. Handschriften unterscheiden sich zu stark, als dass sich mit wenigen festen Regeln alle Varianten beschreiben ließen.

Mit dem **k-Nearest-Neighbors**-Verfahren haben wir einen ersten Algorithmus kennengelernt, der Entscheidungen anhand bereits bekannter Beispiele trifft. Dieses Verfahren benötigt kaum Training, wird jedoch bei großen Datensätzen langsam, da für jede neue Eingabe viele Vergleiche durchgeführt werden müssen.

Das Perzeptron führte anschließend eine neue Idee ein: Anstatt Beispiele direkt zu vergleichen, werden Gewichte gelernt, die eine Entscheidung ermöglichen. Mehrere Perzeptronen können zu einem neuronalen Netz verbunden werden, das komplexere Zusammenhänge beschreiben kann.

Ein wichtiger Schritt war die Einführung nichtlinearer Aktivierungsfunktionen. Erst durch diese Nichtlinearität können mehrere Schichten zusammen komplexe Entscheidungsgrenzen bilden. Ohne Aktivierungsfunktionen wäre auch ein tiefes Netzwerk letztlich nur ein lineares Modell.

Anschließend haben wir das Training neuronaler Netze betrachtet. Mithilfe einer Verlustfunktion wird gemessen, wie gut ein Modell arbeitet. Der Gradient gibt an, in welche Richtung die Gewichte verändert werden müssen, damit der Fehler kleiner wird. Dieser Prozess wird während des Trainings viele Male wiederholt.

Für Bilddaten haben wir schließlich Convolutional Neural Networks (CNNs) kennengelernt. Im Gegensatz zu vollständig verbundenen Netzwerken berücksichtigen CNNs die räumliche Struktur eines Bildes. Dadurch können sie Merkmale wie Kanten, Formen und schließlich ganze Objekte effizient lernen.

Die praktischen Experimente haben gezeigt, dass verschiedene Parameter einen deutlichen Einfluss auf das Verhalten eines Modells haben. Dazu gehören unter anderem:

- die Anzahl der Schichten,
- die Anzahl der Neuronen,
- die Aktivierungsfunktion,

- die Lernrate,
- der Einsatz von Dropout,
- sowie die Wahl der Netzwerkarchitektur.

Dabei wurde deutlich, dass es selten eine einzelne “beste” Einstellung gibt. Stattdessen müssen Architektur, Trainingsverfahren und Datensatz aufeinander abgestimmt werden.

Ausblick

Die Handschrifterkennung ist ein vergleichsweise einfaches Beispiel für Bildklassifikation. Die gleichen Grundideen werden heute in vielen weiteren Bereichen eingesetzt, beispielsweise bei

- der medizinischen Bildanalyse,
- der Objekterkennung in autonomen Fahrzeugen,
- der Qualitätskontrolle in der Industrie,
- der Gesichtserkennung,
- sowie der Analyse von Satelliten- und Luftbildern.

Moderne Modelle besitzen häufig Millionen oder sogar Milliarden von Parametern und werden auf sehr großen Datensätzen trainiert. Trotz dieser Größenordnung beruhen sie auf denselben grundlegenden Konzepten, die wir in diesem Kurs kennengelernt haben:

- Bilder werden als Zahlen dargestellt.
- Modelle lernen aus Beispielen.
- Gewichte werden durch Optimierungsverfahren angepasst.
- Komplexe Aufgaben lassen sich durch viele einfache Rechenschritte lösen.

Die Methoden der künstlichen Intelligenz entwickeln sich kontinuierlich weiter. Gleichzeitig bleibt es wichtig, die Ergebnisse solcher Systeme kritisch zu bewerten. Kein Modell arbeitet fehlerfrei, und die Qualität einer Vorhersage hängt immer von den verfügbaren Daten und dem gewählten Modell ab.

Ein grundlegendes Verständnis der Funktionsweise solcher Verfahren hilft dabei, ihre Möglichkeiten und ihre Grenzen besser einzuordnen.

Anhang

5.1 Die Ableitung

Bevor der Gradient eingeführt werden kann, wird zunächst die Ableitung einer Funktion mit nur einer Variablen betrachtet.

Sei

$$f : \mathbb{R} \rightarrow \mathbb{R}.$$

Die Ableitung von f an der Stelle x ist definiert als

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Der Differenzenquotient

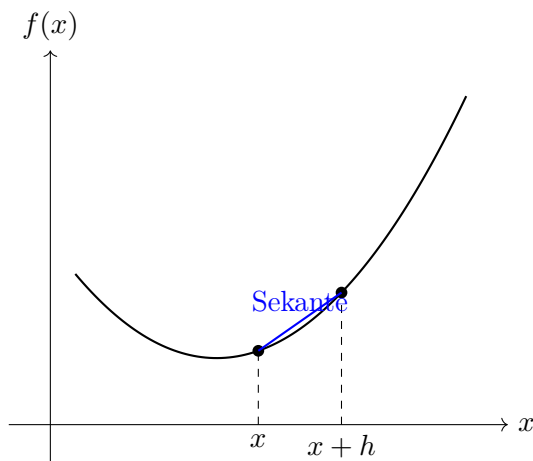
$$\frac{f(x+h) - f(x)}{h}$$

beschreibt die Steigung der Sekante durch die beiden Punkte $(x, f(x))$ und $(x+h, f(x+h))$. Lässt man den Abstand h gegen Null gehen, so geht die Sekante in die Tangente über. Die Ableitung entspricht somit der Steigung der Tangente an den Graphen der Funktion.

Ist die Ableitung positiv, so steigt die Funktion an dieser Stelle. Ist sie negativ, so fällt die Funktion. An einem lokalen Minimum oder Maximum gilt häufig

$$f'(x) = 0.$$

Die Idee der Ableitung bildet die Grundlage des Gradienten, der später für Funktionen mit mehreren Variablen eingeführt wird.



5.2 Der Gradient

Bei Funktionen mit nur einer Variablen beschreibt die Ableitung, wie stark sich der Funktionswert bei einer kleinen Änderung der Eingabe verändert. Besitzt eine Funktion mehrere Variablen, so

kann sich jede Variable unabhängig von den anderen ändern. In diesem Fall betrachtet man zunächst die Änderung in jeder Koordinatenrichtung einzeln.

Sei

$$f : \mathbb{R}^2 \rightarrow \mathbb{R},$$

also eine Funktion mit zwei Eingangsgrößen x und y . Die partiellen Ableitungen werden definiert als

$$\frac{\partial f}{\partial x}(x, y) = \lim_{h \rightarrow 0} \frac{f(x + h, y) - f(x, y)}{h},$$

und

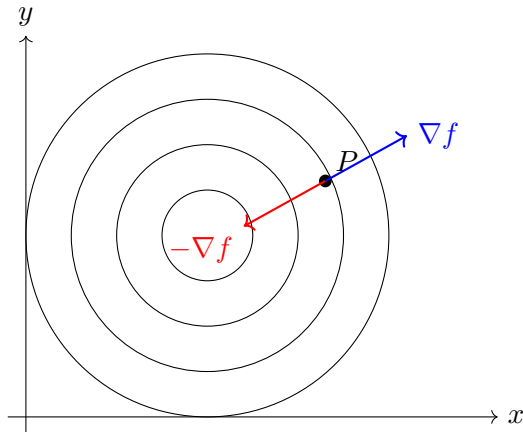
$$\frac{\partial f}{\partial y}(x, y) = \lim_{h \rightarrow 0} \frac{f(x, y + h) - f(x, y)}{h}.$$

Die partielle Ableitung nach x misst also die Änderung der Funktion, wenn nur x verändert wird. Analog beschreibt die partielle Ableitung nach y die Änderung in y -Richtung.

Fasst man beide partiellen Ableitungen zu einem Vektor zusammen, erhält man den *Gradienten*

$$\nabla f(x, y) = \begin{pmatrix} \frac{\partial f}{\partial x}(x, y) \\ \frac{\partial f}{\partial y}(x, y) \end{pmatrix}.$$

Der Gradient zeigt immer in die Richtung des stärksten Anstiegs der Funktion. Soll ein Minimum gefunden werden, bewegt man sich daher in die entgegengesetzte Richtung des Gradienten. Dieses Prinzip bildet die Grundlage des sogenannten *Gradient Descent*, der später zum Trainieren neuronaler Netze verwendet wird.



5.3 Lineare Entscheidungsgrenzen und die Rolle der Aktivierungsfunktion

Beim Perzeptron müssen wir zwischen zwei verschiedenen Konzepten unterscheiden:

1. die mathematische Berechnung innerhalb des Neurons,
2. die Form der Entscheidungsgrenze für die Klassifikation.

Das Perzeptron berechnet zunächst eine gewichtete Summe:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

oder kürzer:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Diese Berechnung ist eine lineare Kombination der Eingaben.
Danach wird eine Aktivierungsfunktion angewendet:

$$y = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

Diese Funktion nennt man **Schwellwertfunktion** oder **Sprungfunktion**.

5.3.1 Perzeptron und linearität

Die Sprungfunktion selbst ist mathematisch nicht linear.
Sie erzeugt eine harte Entscheidung:

$$\text{Eingabe} \rightarrow 0 \text{ oder } 1$$

Trotzdem wird das Perzeptron als **linearer Klassifikator** bezeichnet.
Der Grund ist die Form der Grenze zwischen den beiden Klassen.
Die Grenze entsteht dort, wo:

$$z = 0$$

also:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Bei zwei Eingaben ergibt sich:

$$w_1 x_1 + w_2 x_2 + b = 0$$

Diese Gleichung beschreibt eine Gerade.

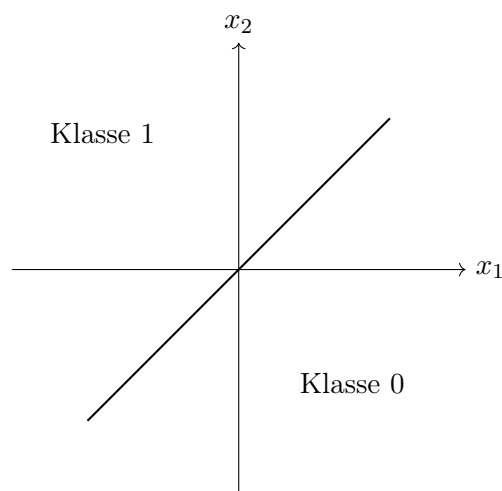


Abbildung 5.1: Ein Perzeptron kann Klassen durch eine lineare Grenze trennen.

Für einfache Probleme reicht eine solche Grenze aus.
Beispielsweise:

Ist der Punkt links oder rechts von einer Geraden?

5.3.2 Das XOR Problem

Bilder enthalten jedoch sehr komplexe Muster.

Bei Handschriften können Klassen beliebig geformte Bereiche im Merkmalsraum bilden.

Eine einfache Gerade kann beispielsweise nicht das folgende Problem lösen:

XOR:

x_1	x_2	Klasse
0	0	0
0	1	1
1	0	1
1	1	0

Keine einzelne Gerade kann diese vier Punkte korrekt trennen.

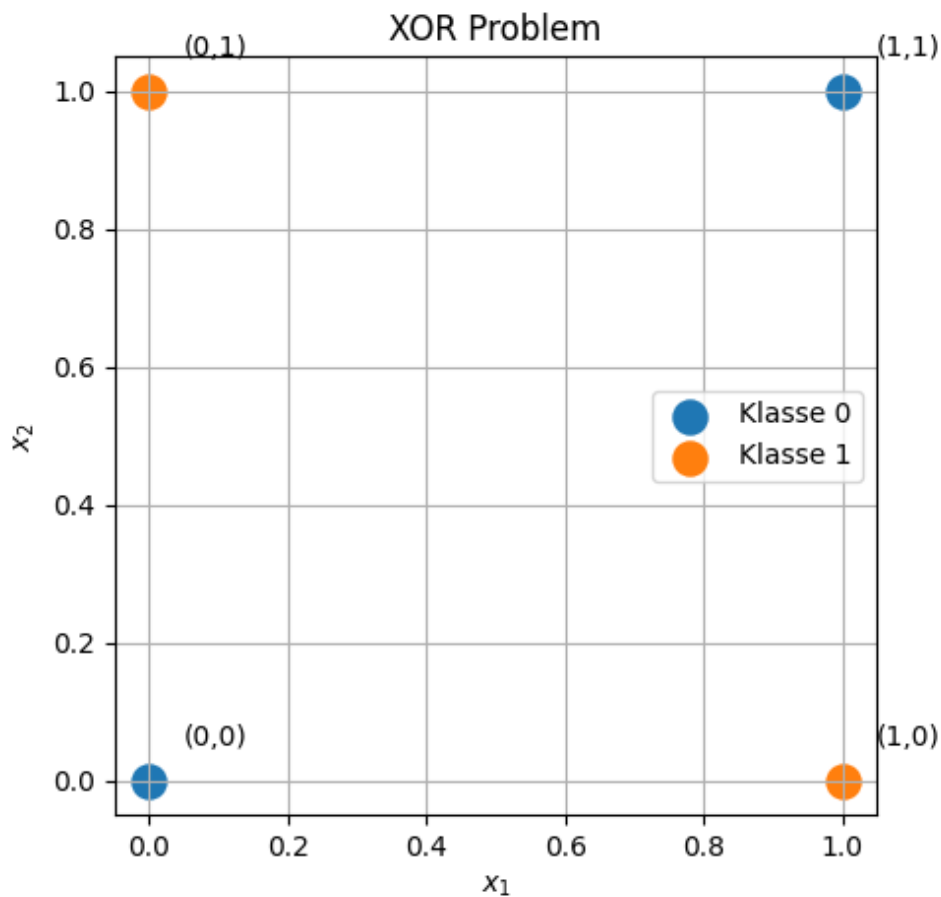


Abbildung 5.2: Visualisierung XOR problem

Lineare Aktivierungsfunktionen Man könnte mehrere Perzeptronen hintereinander verbinden:

Schicht 1 $\rightarrow$ Schicht 2 $\rightarrow$ Schicht 3

Wenn jedoch jede Schicht nur lineare Berechnungen durchführt:

$$y = W_3(W_2(W_1x))$$

kann man die Matrizen zusammenfassen:

$$y = Wx$$

Das gesamte Netzwerk wäre wieder nur eine einzige lineare Funktion. Mehr Schichten würden also keinen zusätzlichen Nutzen bringen.

$$\boxed{\text{Linear} + \text{Linear} + \text{Linear} = \text{Linear}}$$

Damit neuronale Netze komplexe Muster lernen können, benötigen sie eine nichtlineare Aktivierungsfunktion.

Nichtlineare Aktivierungsfunktionen Eine Aktivierungsfunktion verändert die Ausgabe eines Neurons:

$$a = f(z)$$

Beispiele sind:

Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Die Ausgabe liegt zwischen 0 und 1:

$$0 < \sigma(z) < 1$$

ReLU Heute wird häufig die Funktion ReLU verwendet:

$$\text{ReLU}(z) = \max(0, z)$$

Sie lässt positive Werte unverändert und setzt negative Werte auf null. Durch diese Nichtlinearität kann jede Schicht neue Eigenschaften lernen:

$$\begin{aligned} \text{Pixel} &\rightarrow \text{Kanten} \\ &\rightarrow \text{Formen} \\ &\rightarrow \text{Ziffern} \end{aligned}$$

Die wichtige Erkenntnis lautet:

Ein einzelnes Perzeptron berechnet eine lineare Kombination von Eingaben. Die Nichtlinearität zwischen den Schichten ermöglicht es neuronalen Netzen, komplexe Muster zu lernen.

5.4 Backpropagation in neuronalen Netzen

Das Training eines neuronalen Netzes besteht darin, die Gewichte des Netzwerks so anzupassen, dass die Vorhersagen möglichst gut mit den gewünschten Ausgaben übereinstimmen. Dazu wird zunächst ein Fehlermaß, die sogenannte Verlustfunktion (*Loss Function*), definiert.

Sei y die gewünschte Ausgabe und $\hat{y}$ die vom Netzwerk berechnete Ausgabe. Eine häufig verwendete Verlustfunktion für Klassifikationsprobleme ist beispielsweise der quadratische Fehler:

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2.$$

Das Ziel des Trainings besteht darin, die Gewichte

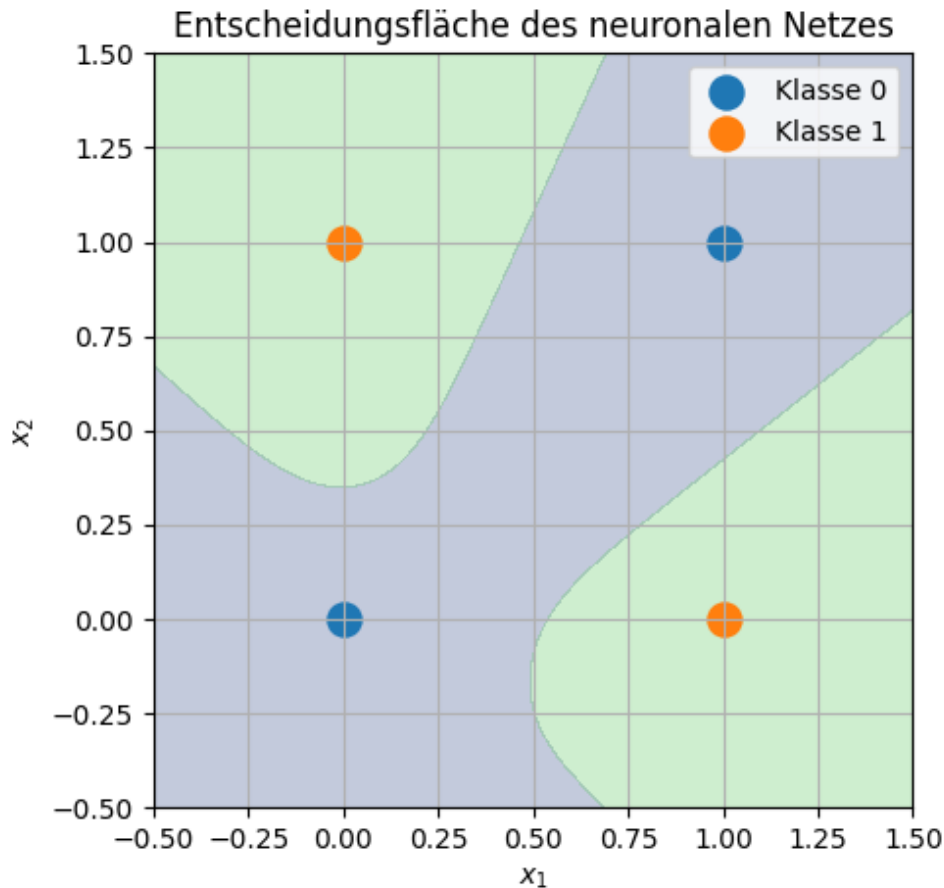


Abbildung 5.3: Entscheidungsfläche des neuronalen Netzes

$$W_1, W_2, \dots, W_n$$

so zu verändern, dass die Verlustfunktion minimiert wird:

$$\min_W L(W).$$

Dazu muss bestimmt werden, welchen Einfluss jedes einzelne Gewicht auf den Fehler besitzt. Diese Information liefert der Gradient der Verlustfunktion:

$$\nabla_W L = \frac{\partial L}{\partial W}.$$

Die Gewichte werden anschließend mithilfe des Gradientenabstiegs aktualisiert:

$$W_{\text{neu}} = W_{\text{alt}} - \eta \frac{\partial L}{\partial W},$$

wobei η die Lernrate bezeichnet. Sie bestimmt die Größe der einzelnen Änderungsschritte.

5.4.1 Die Kettenregel als Grundlage

Die zentrale Idee der Backpropagation besteht darin, den Einfluss jedes Gewichts auf den Fehler effizient zu berechnen. Da ein neuronales Netzwerk aus vielen aufeinanderfolgenden Funktionen besteht, wird dazu die Kettenregel der Differentialrechnung verwendet.

Betrachtet man beispielsweise zwei aufeinanderfolgende Berechnungen

$$z = f(x), \quad y = g(z),$$

so ergibt sich die Änderung von y bezüglich x durch

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial z} \frac{\partial z}{\partial x}.$$

Diese Regel erlaubt es, den Fehler vom Ausgang des Netzwerks schrittweise zurück zu den einzelnen Schichten zu übertragen.

5.4.2 Vorwärts- und Rückwärtsdurchlauf

Während des Vorwärtsdurchlaufs (*Forward Pass*) werden die Eingaben durch das Netzwerk propagiert. Jede Schicht berechnet eine Aktivierung:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)},$$

$$a^{(l)} = \sigma(z^{(l)}),$$

wobei $W^{(l)}$ die Gewichtsmatrix, $b^{(l)}$ der Bias-Vektor und σ die Aktivierungsfunktion der Schicht l ist.

Am Ende des Vorwärtsdurchlaufs wird die Verlustfunktion berechnet.

Beim Rückwärtsdurchlauf (*Backward Pass*) wird der Fehler vom Ausgang des Netzwerks zurück zu den vorherigen Schichten übertragen. Für jede Schicht wird dabei berechnet, wie stark ihre Gewichte zum Gesamtfehler beitragen.

Für eine Gewichtsmatrix einer Schicht gilt:

$$\frac{\partial L}{\partial W^{(l)}}$$

als Maß dafür, wie eine kleine Änderung der Gewichte den Fehler beeinflusst.

Anschließend werden alle Gewichte aktualisiert:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}.$$

Dieser Vorgang wird für viele Trainingsbeispiele und zahlreiche Trainingsdurchläufe (*Epochen*) wiederholt. Mit jedem Schritt werden die Gewichte so angepasst, dass das Netzwerk immer bessere Vorhersagen erzeugt.

Die Backpropagation ist somit kein Lernalgorithmus im eigentlichen Sinne, sondern ein effizientes Verfahren zur Berechnung der Gradienten, die für die Optimierung der Netzwerkparameter benötigt werden.

Kapitel 6

Literaturverzeichnis

- [1] Yoshua Bengio, Ian Goodfellow, Aaron Courville, et al. *Deep learning*, volume 1. MIT press Cambridge, MA, USA, 2017.
- [2] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.
- [3] Aurélien Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. Ö'Reilly Media, Inc.", 2022.
- [4] Trevor Hastie, Robert Tibshirani, Jerome Friedman, et al. The elements of statistical learning, 2009.
- [5] Michael A Nielsen. *Neural networks and deep learning*, volume 25. Determination press San Francisco, CA, USA, 2015.
- [6] Sebastian Raschka and Vahid Mirjalili. *Python machine learning: Machine learning and deep learning with Python, scikit-learn, and TensorFlow 2*. Packt publishing ltd, 2019.